



# Machine Learning

BirdCLEF+ 2026

Paul Béglin  
Ekaterina Bogush

# Contents

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                   | <b>2</b>  |
| <b>2</b>  | <b>Dataset description</b>                            | <b>2</b>  |
| 2.1       | Clean audio . . . . .                                 | 2         |
| 2.2       | Soundscapes . . . . .                                 | 2         |
| 2.3       | Taxonomy . . . . .                                    | 3         |
| <b>3</b>  | <b>Exploratory Data Analysis (EDA)</b>                | <b>3</b>  |
| 3.1       | Clean audio . . . . .                                 | 3         |
| 3.2       | Soundscapes . . . . .                                 | 6         |
| 3.3       | Taxonomy . . . . .                                    | 7         |
| <b>4</b>  | <b>Problem statement</b>                              | <b>7</b>  |
| <b>5</b>  | <b>Proposed approach</b>                              | <b>7</b>  |
| 5.1       | Overview of pipeline . . . . .                        | 7         |
| 5.2       | Clean audio training . . . . .                        | 7         |
| 5.3       | Taxonomy-aware modeling . . . . .                     | 7         |
| 5.4       | Models variants . . . . .                             | 8         |
| 5.5       | Soundscapes adaptation . . . . .                      | 8         |
| <b>6</b>  | <b>Audio preprocessing</b>                            | <b>8</b>  |
| 6.1       | Clean audio duration analysis and filtering . . . . . | 8         |
| 6.2       | Soft spectral gating . . . . .                        | 9         |
| 6.3       | Clean audio silence trimming . . . . .                | 11        |
| <b>7</b>  | <b>Feature engineering</b>                            | <b>12</b> |
| 7.1       | Audio segmentation and temporal aggregation . . . . . | 12        |
| 7.2       | Mel-spectrograms . . . . .                            | 12        |
| 7.3       | Spectral dynamics . . . . .                           | 13        |
| 7.4       | Spectral shape and temporal activity . . . . .        | 14        |
| 7.5       | Texture analysis . . . . .                            | 17        |
| 7.6       | Species Mel profiles . . . . .                        | 19        |
| 7.7       | Reducing the number of Mel bins . . . . .             | 21        |
| 7.8       | Dimensionality reduction . . . . .                    | 22        |
| 7.9       | Soundscapes context features . . . . .                | 25        |
| <b>8</b>  | <b>Experimental details</b>                           | <b>25</b> |
| 8.1       | Dataset dimensionality . . . . .                      | 25        |
| 8.2       | Train/validation split . . . . .                      | 25        |
| 8.3       | Evaluation metrics . . . . .                          | 25        |
| 8.4       | Model training . . . . .                              | 26        |
| 8.5       | Model architectures and training strategies . . . . . | 27        |
| <b>9</b>  | <b>Evaluation</b>                                     | <b>29</b> |
| 9.1       | Evaluation on clean audio . . . . .                   | 29        |
| 9.2       | Evaluation on soundscapes . . . . .                   | 31        |
| <b>10</b> | <b>Conclusion</b>                                     | <b>33</b> |
|           | <b>Appendix</b>                                       | <b>34</b> |

# 1 Introduction

The objective of this project is to investigate the BirdCLEF+ 2026 dataset and evaluate the performance of classical machine learning approaches under a strict no-deep-learning constraint. While convolutional neural networks (CNNs) are typically among the top-performing models on the BirdCLEF Kaggle leaderboard, we instead focus on assessing how far classical methods can go on this task.

The BirdCLEF+ 2026 challenge consists of predicting the presence of 234 different species in ecological audio recordings. In particular, the task becomes more challenging in noisy soundscapes, where multiple species may be present simultaneously.

## 2 Dataset description

Kaggle provides three datasets:

1. Clean audio recordings with associated metadata in `.csv` format, where usually a single species is clearly audible.
2. Soundscape audio recordings with corresponding metadata in `.csv` format.
3. A two-level taxonomy (species and higher-level class) for each target species.

After removing duplicate rows, we obtain the following statistics:

| Dataset      | # Downloaded Audios | # Metadata Rows | Annotated Audios                |
|--------------|---------------------|-----------------|---------------------------------|
| Clean audios | 35,549              | 35,549          | 35,549                          |
| Soundscapes  | 10,658              | 66              | 66 (or 739 chunks of 5 seconds) |

Table 1: Overview of the dataset after preprocessing

### 2.1 Clean audio

Table 3 summarizes the metadata fields associated with the clean audio. The clean audio dataset contains recordings that are generally focused and are therefore annotated with a single dominant species, although additional background species may still be present. Each species may be represented by multiple recordings.

| Feature                       | Type   | Missing (%) | # Unique values | Additional stats       | Comment                                      |
|-------------------------------|--------|-------------|-----------------|------------------------|----------------------------------------------|
| <code>primary_label</code>    | string | 0.00        | 206             | -                      | Main species label (target variable)         |
| <code>secondary_labels</code> | list   | -           | 156             | avg: 0.21, max len: 15 | Additional co-occurring species (incomplete) |
| <code>type</code>             | list   | -           | 580             | avg: 0.75, max: 7      |                                              |
| <code>latitude</code>         | float  | 0.00        | 15,270          | -                      | Geographical latitude                        |
| <code>longitude</code>        | float  | 0.00        | 15,265          | -                      | Geographical longitude                       |
| <code>scientific_name</code>  | string | 0.00        | 206             | -                      | Latin species name                           |
| <code>common_name</code>      | string | 0.00        | 206             | -                      | Common species name                          |
| <code>class_name</code>       | string | 0.00        | 5               | -                      | High-level taxonomic class                   |
| <code>inat_taxon_id</code>    | int    | 0.00        | 206             | -                      | iNaturalist taxon identifier                 |
| <code>author</code>           | string | 3.52        | 4,016           | -                      | Contributor identifier                       |
| <code>license</code>          | string | 0.00        | 7               | -                      | Usage license                                |
| <code>rating</code>           | float  | 36.14       | 10              | -                      | Audio quality / community rating (sparse)    |
| <code>url</code>              | string | 0.00        | 35,549          | -                      |                                              |
| <code>filename</code>         | string | 0.00        | 35,549          | -                      | Audio file identifier                        |
| <code>collection</code>       | string | 0.00        | 2               | -                      | Either Xeno-canto or iNaturalist             |

Table 2: Clean audio metadata schema overview

### 2.2 Soundscapes

Table 3 summarizes the metadata fields associated with the soundscape recordings. Soundscape recordings correspond to continuous ecological audio, each lasting one minute. They are partially annotated in 5-second chunks, meaning that some segments are left unannotated. Each chunk may contain multiple simultaneously active species.

In addition to the audio content, metadata is extracted from the filenames following the Kaggle dataset specification. This includes temporal and geolocation information such as recording time and recording site.

| Feature                         | Type     | Missing (%) | # Unique values | Additional stats           | Comment                            |
|---------------------------------|----------|-------------|-----------------|----------------------------|------------------------------------|
| <code>filename</code>           | string   | 0.00        | 66              | -                          | Soundscape file identifier         |
| <code>start</code>              | string   | 0.00        | 12              | -                          | Segment start time                 |
| <code>end</code>                | string   | 0.00        | 12              | -                          | Segment end time                   |
| <code>primary_label_list</code> | list     | -           | 75              | avg: 4.22, min: 1, max: 10 | Multi-label annotation per segment |
| <code>datetime</code>           | datetime | 0.00        | 62              | -                          | Recording timestamp                |
| <code>site</code>               | int      | 0.00        | 9               | -                          | Recording site identifier          |

Table 3: Soundscape metadata overview

## 2.3 Taxonomy

Table 4 summarizes the taxonomy structure. The taxonomy dataset defines the full set of target classes used in this task. In total, it contains 234 species-level labels (i.e., possible values of `primary_label`). Among them, 206 species are present in the clean audio dataset, while only 75 species appear in the annotated soundscape segments.

A notable issue is that 28 species (including Guaraní leaf-litter frog, *Chiasmocleis mehelyi*, Southern Orange-legged Leaf Frog, and 25 insect sonotypes) do not have any corresponding clean training recordings. These classes appear only in soundscapes, and, hence, no clean training samples to rely on.

| Feature                      | Type   | Missing (%) | # Unique values | Additional stats | Comment                    |
|------------------------------|--------|-------------|-----------------|------------------|----------------------------|
| <code>primary_label</code>   | string | 0.00        | 234             | -                | Target class label         |
| <code>inat_taxon_id</code>   | int    | 0.00        | 210             | -                | iNaturalist identifier     |
| <code>scientific_name</code> | string | 0.00        | 234             | -                | Latin species name         |
| <code>common_name</code>     | string | 0.00        | 234             | -                | Human-readable name        |
| <code>class_name</code>      | string | 0.00        | 5               | -                | High-level taxonomic class |

Table 4: Taxonomy metadata overview

## 3 Exploratory Data Analysis (EDA)

In this section, we perform an exploratory data analysis of the datasets to understand the distribution of key variables and their interactions, and to identify potential challenges for predictive modeling that must be taken into account.

### 3.1 Clean audio

Figure 1a shows the distribution of species classes. The dataset is heavily imbalanced, with *Aves* accounting for almost 90% of all recordings. Figure 1b illustrates the empirical cumulative distribution function (ECDF)<sup>1</sup> of the number of recordings per `primary_label`, grouped by taxonomic class.

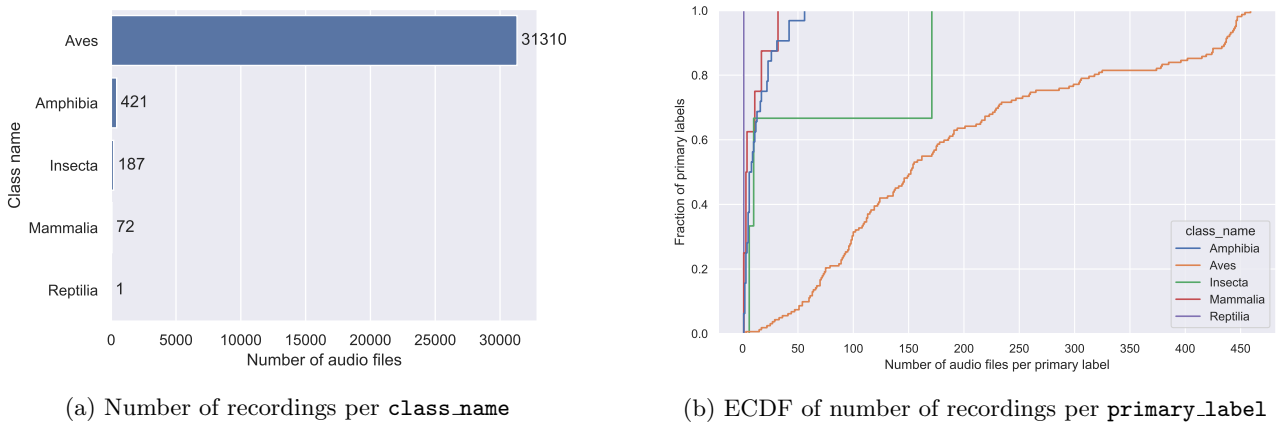


Figure 1: Class distribution analysis for the clean audio dataset. The figures highlight the strong imbalance in the number of recordings across species

<sup>1</sup>We use the ECDF instead of standard bar plots due to the presence of 206 distinct classes; bar plots would quickly become unreadable and less informative.

All species belonging to the Mammalia and Amphibia classes contain fewer than 50 recordings. Within the Insecta class, two species contain fewer than 50 recordings, while one species contains approximately 175 recordings. The distribution of recordings in the Aves class is highly skewed: 50% of bird species contain fewer than 150 recordings, whereas approximately 20% contain more than 350 recordings.

Figure 2 illustrates the geographical distribution of recordings across species. Clean audio recordings originate from a wide range of global locations, whereas soundscape recordings are concentrated in a much more restricted geographical region. This geographical narrowness of the soundscapes should be kept in mind. Kaggle also warns about possible regional variations (dialects) in some species vocalizations; therefore, a model trained primarily on soundscapes may not generalize well to other locations.

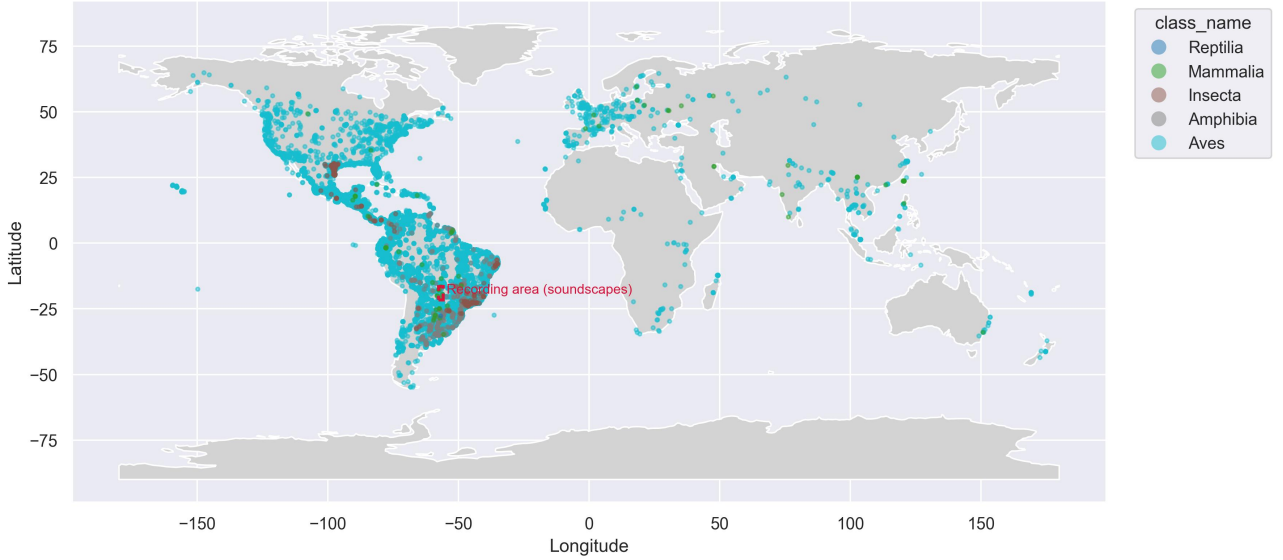


Figure 2: Geographical distribution of recordings across species for clean audio and soundscape datasets

We also investigated the behavior of ratings and secondary labels, although both sources are incomplete (ratings are not provided for all recordings in iNaturalist, and Kaggle notes that secondary labels may be incomplete). Figure 3 shows the distribution of audio ratings provided by users of the Xeno-canto website.

Figure 4 illustrates the relationship between recording rating and the number of secondary labels. We observe that each recording is assigned a base rating, which is reduced in steps of 0.5 as specified in the Kaggle guidelines. However, the rating appears largely independent of the number of secondary labels, as high-quality recordings (e.g., 4.5) can still contain 6-8 additional species.

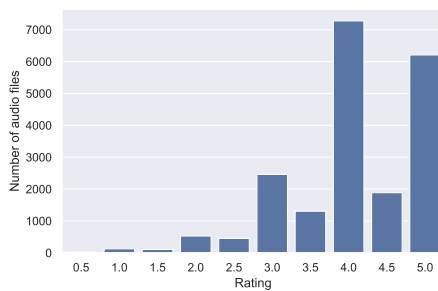


Figure 3: Distribution of recording ratings in the dataset

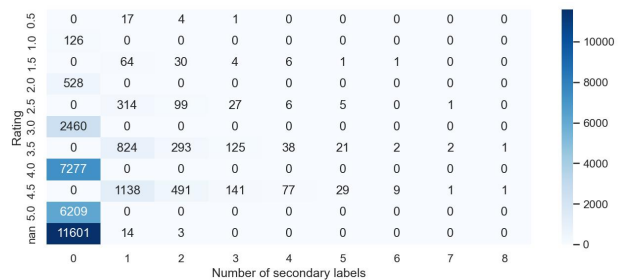


Figure 4: Relationship between recording rating and number of secondary labels

Figure 5 illustrates the co-occurrence patterns between the number of secondary labels and the 50 most frequent label types. Unfortunately, the `type` field is not normalized (e.g., variations such as *call/calls* or *canto/canto em voo*), and some types that implicitly indicate the presence of multiple species (e.g., duet song) can still have zero secondary labels. This further highlights the incompleteness of the secondary label annotations.

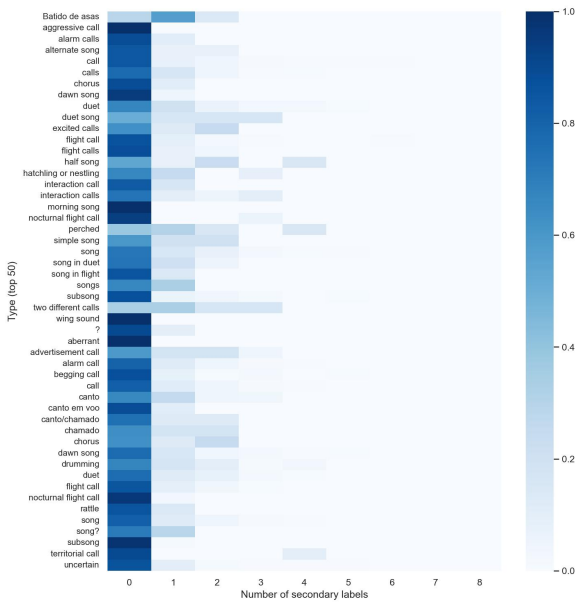


Figure 5: Co-occurrence between secondary label counts and the 50 most frequent label types

Figure 6 shows, for each `class_name`, a randomly sampled recording. For each recording, we display the waveform together with different audio representations: linear spectrogram, Mel-spectrogram (128 Mel bins), and MFCC (13 coefficients). All spectrograms are shown on a dB scale with `ref=1.0`, allowing comparison of relative signal intensity across recordings. This helps assess whether some recordings are inherently quieter than others. Notably, different species appear to occupy distinct frequency bands, with certain frequency ranges being more prominent for some species and less active for others.

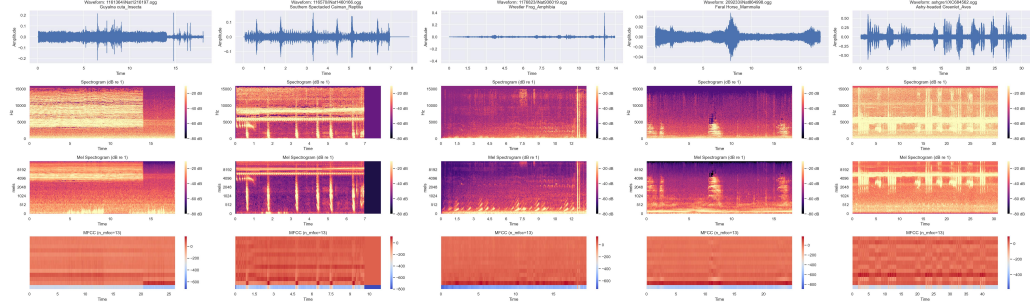


Figure 6: Waveform and spectral representations (linear spectrogram, Mel-spectrogram, MFCC) for randomly sampled recordings per `class_name`

Figure 7 shows multiple recordings corresponding to the same `primary_label`. Similar spectral patterns are observed across the different spectrogram representations, indicating consistent acoustic characteristics within the class. However, recordings 3 and 5 (counting from the left) appear noticeably quieter than recordings 1, 2, and 4.

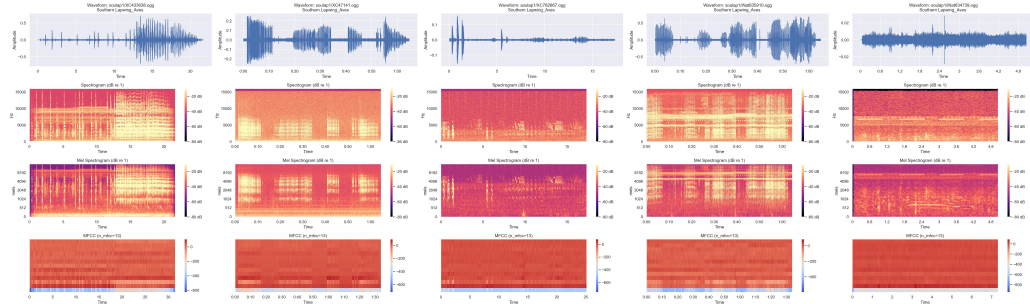


Figure 7: Multiple recordings for a single `primary_label` showing consistent spectral patterns and variations in signal intensity

### 3.2 Soundscapes

Figure 8 illustrates the distribution of 75 `primary_labels`, colored by `class_name`. In contrast to the clean audio dataset, *Amphibia* dominate the soundscape recordings, while *Aves* represent a smaller proportion.

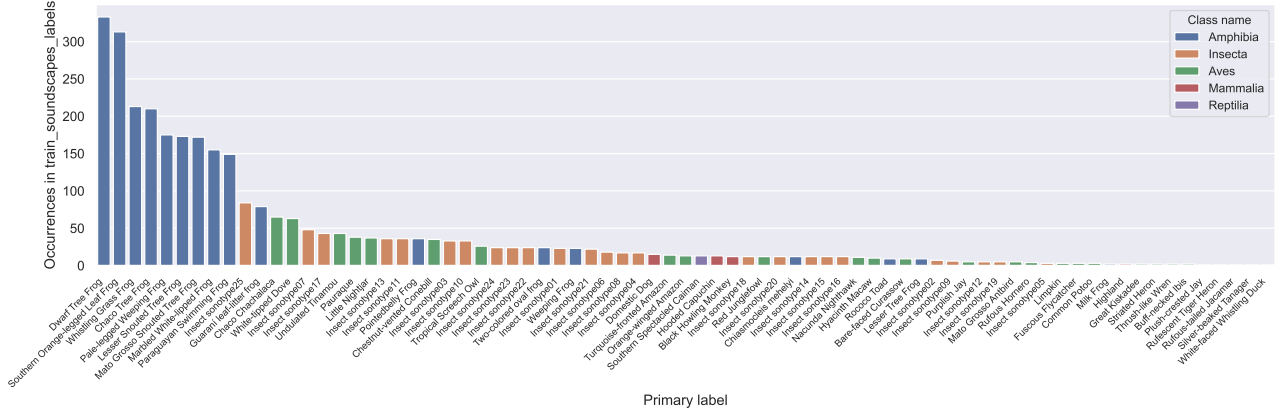


Figure 8: Distribution of primary labels in soundscape recordings, colored by taxonomic class

Figure 9 shows the species occurrence tiMeline for four selected soundscape recordings. We observe that some segments contain no annotated species at all. Section A in the Appendix provides annotated tiMelines for all 66 soundscapes.

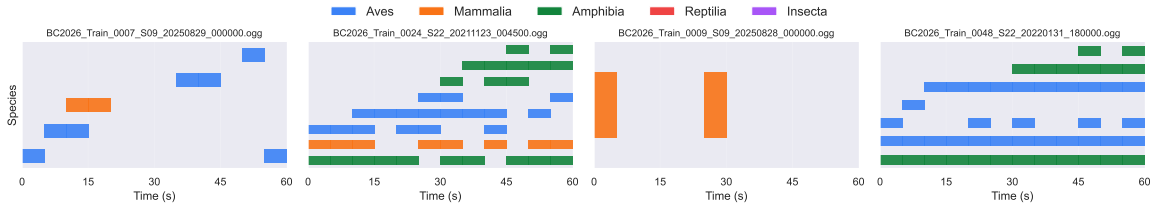


Figure 9: Species occurrence tiMelines for selected soundscape recordings, showing annotated and unannotated segments

Figure 10 shows species occurrence patterns by hour, month (seasonality), and recording site. Due to class imbalance, the heatmaps are normalized per species and should be interpreted column-wise. We also cap values at 0.3, since some species are strongly associated with specific months or sites, which would otherwise dominate the visualization and obscure weaker patterns.

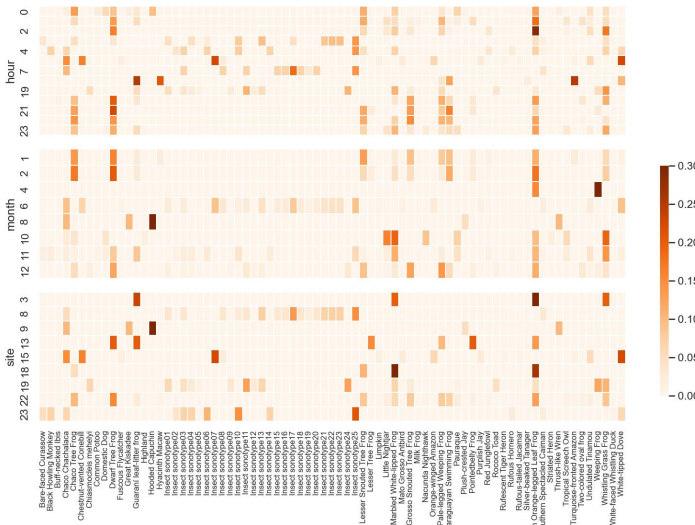


Figure 10: Species occurrence patterns by hour, month, and recording site using species-wise normalized heatmaps

We observe clear temporal and spatial structuring in species occurrences. Some species are predominantly nocturnal and are rarely observed during daytime hours, while others are mainly active during the day. Similar patterns are visible across months, where certain species are more frequent during summer and others during winter, indicating seasonal effects. A comparable dependency is observed with respect to recording sites, where some species are strongly associated with specific locations, reflecting possible habitat preferences.

### 3.3 Taxonomy

Figure 11 shows the taxonomic hierarchy and the relative number of species in each class. *Aves* contains the largest number of species. The *Insecta* class is largely composed of sonotypes that appear only in soundscapes and are absent from the clean audio dataset. *Reptilia* is represented by a single species.

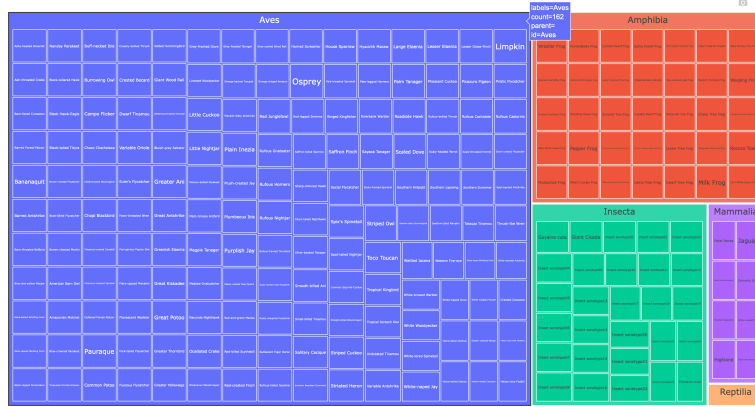


Figure 11: Taxonomic distribution of species across classes, showing the number of species per taxonomic group

## 4 Problem statement

Our primary objective is to predict `primary_label` in soundscape recordings. However, the soundscape dataset contains only 739 annotated 5-second chunks and covers only 75 species out of 234, with recordings concentrated in a narrow geographical region. This makes it insufficient to train a robust and generalizable model directly.

To mitigate this limitation, we instead rely primarily on the clean audio dataset, which contains approximately 35k recordings covering 206 species. The goal is to learn species-specific acoustic representations from cleaner recordings and transfer them to the soundscape domain.

## 5 Proposed approach

### 5.1 Overview of pipeline

Our approach follows a two-stage pipeline. We first train models on clean audio recordings to learn species-specific acoustic representations. These models are then adapted to the soundscape domain, where recordings are noisier and more complex.

Regarding input features, we rely only on audio and taxonomic structure, as other metadata variables are either highly sparse (e.g., rating) or not informative for species identification (e.g., author information).

### 5.2 Clean audio training

We treat the prediction of `primary_label` on the clean audio dataset as a single-label multiclass classification problem. Initially, we considered incorporating secondary labels as additional targets, which would reformulate the task as a multi-label classification problem. However, due to their sparsity, incompleteness, and uncertain reliability, we do not include them in the training objective.

We adopt a One-vs-Rest (OvR) strategy to transition from a single-label training setting to a multi-label formulation. This allows the model to produce independent probabilities for each target.

### 5.3 Taxonomy-aware modeling

To leverage the taxonomic structure of the dataset, we avoid training a single classifier over all 206 species, which would require learning complex decision boundaries between both acoustically similar species, e.g. closely

related Aves, and very dissimilar ones, e.g. Insecta vs Mammalia, while also being affected by strong class imbalance.

Instead, we adopt a hierarchical modeling strategy. We first train a generalist model to predict `class_name`, corresponding to the taxonomic class. Then, we train a set of specialist models, each restricted to a single taxonomic class, and responsible for predicting `primary_label` within that subset. This decomposition reduces intra-class variability and simplifies the classification task, since each specialist model only needs to distinguish between acoustically similar species within the same taxonomic group.

This design results in one global model for taxonomic classification and multiple local models for fine-grained species recognition. At inference time, both levels are combined probabilistically. The model outputs estimates for  $\mathbb{P}(\text{class} = c)$  and  $\mathbb{P}(\text{label} = y \mid \text{class} = c)$ , which are combined as:

$$\begin{aligned}\mathbb{P}(\text{label} = y) &= \sum_c \mathbb{P}(\text{label} = y, \text{class} = c) \\ &= \sum_c \mathbb{P}(\text{label} = y \mid \text{class} = c) \mathbb{P}(\text{class} = c) \\ &= \mathbb{P}(\text{label} = y \mid \text{class} = c_y) \mathbb{P}(\text{class} = c_y) \\ &= \exp(\log P(\text{label} = y \mid \text{class} = c_y) + \log P(\text{class} = c_y)) \text{ for numerical stability}\end{aligned}$$

with  $\mathbb{P}(\text{label} = y \mid \text{class} = c') = 0$  for  $c' \neq c_y$ ,  
since each label  $y$  belongs to exactly one class.

## 5.4 Models variants

We explore two different training strategies. The first one relies on segmenting clean audio into fixed 5-second chunks, which matches the annotation granularity of soundscapes. The second one follows a Multiple Instance Learning (MIL) formulation, where recordings are split into 1-second chunks, and predictions are later aggregated to obtain a 5-second chunk-level output.

We also initially considered a third approach without explicit segmentation, where features are aggregated globally over the entire recording. However, we’ve decided to discard this strategy, as global pooling can overly smooth the temporal structure of the signal. Since vocalizations of some species are sparse in time, such averaging may remove discriminative cues and limit the model’s ability to learn meaningful decision boundaries that generalize to soundscapes.

## 5.5 Soundscapes adaptation

To adapt the model from clean audio to soundscapes, we use a second-stage model that refines predictions from the clean-audio model by incorporating soundscape context features (site and recording time).

# 6 Audio preprocessing

This section describes the preprocessing steps applied to both clean and soundscape recordings prior to feature extraction.

## 6.1 Clean audio duration analysis and filtering

The duration of clean audio is not normalized, unlike soundscape recordings. Figure 12 shows the distribution of audio durations in seconds on a logarithmic scale. We observe strong variability, some recordings are shorter than half a second (these are corrupted files, as verified by listening to the recordings), while others are longer than one hour, where bird vocalizations are often very sparse and appear only in short segments.

To reduce the effect of outliers, we remove recordings shorter than the 5th percentile and longer than the 95th percentile of the duration distribution. Before applying this filter, we checked that no species is heavily affected, meaning that for no `primary_label` more than 80% of its recordings fall in the removed ranges. After filtering, the clean audio dataset contains 31,991 files.

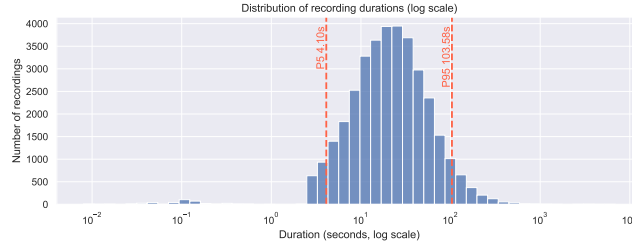


Figure 12: Distribution of clean audio durations (in seconds) on a logarithmic scale

## 6.2 Soft spectral gating

Before extracting any features, and to make them more representative of each species, we apply a noise reduction step to every recording – both soundscapes and clean audio. For that, we use **soft spectral gating** method. The main assumption of this method is that low energy frames are mostly noise. A classical spectral gating suppresses noise by applying a binary mask in the time-frequency domain – components below a noise threshold are zeroed out, those above are kept untouched. To avoid strict binary gating, we replaced it with a mask per  $M(f, t) \in ]0, 1[$ , so each frame is attenuated proportionally to how close its amplitude is to the noise floor. This method represents an approximation of the theoretical optimal Wiener filter.

Soft spectral gating is applied as follows:

1. **Time-frequency decomposition.** The input signal  $y(t)$  is transformed from time into frequency domain via the Short-Time Fourier Transform (STFT), denoted as  $S(f, t)$ .
2. **Noise floor estimation.** The average spectral power of each frame is computed as:

$$E(t) = \frac{1}{F} \sum_f |S(f, t)|^2$$

$$F = \lfloor \frac{n_{fft}}{2} \rfloor + 1 \text{ the number of frequency bins}$$

Frames below the  $p$ -th energy percentile are considered as noise candidates:

$$\mathcal{T} = \{t : E(t) \leq \text{percentile}_p(E)\}.$$

If  $|\mathcal{T}| < 5$ , we avoid applying this method as there are too few frames to reliably estimate noise and the original signal is returned unchanged. Otherwise, the noise magnitude is estimated independently for each frequency bin by taking the median over the noise candidate frames:

$$N(f) = \text{median}_{t \in \mathcal{T}} |S(f, t)|$$

3. **Soft mask computation.** For each time-frequency bin, we define an amplitude-domain signal-to-noise ratio:

$$\text{SNR}(f, t) = \frac{|S(f, t)|}{N(f) + \varepsilon},$$

where  $\varepsilon > 0$  avoids division by zero. The soft gate is then the mask:

$$M(f, t) = \frac{\text{SNR}(f, t)}{\text{SNR}(f, t) + \alpha} \in ]0, 1[, \quad \alpha > 0,$$

where  $\alpha$  controls attenuation strength: a larger  $\alpha$  shifts the threshold upward, so that bins must have a higher SNR to be preserved – resulting in more aggressive denoising. Bins well below the noise floor ( $\text{SNR} < 1$ ) are strongly suppressed, while bins well above it ( $\text{SNR} > 1$ ) are preserved. This avoids abrupt on/off switching of hard spectral gating.

4. **Mask smoothing.** Frame-independent masking can introduce musical noise, which corresponds to short metallic artifacts that may appear after noise reduction. This occurs because the mask is computed independently for each time-frequency bin: some bins may be strongly attenuated while neighbouring bins are preserved, creating isolated residual energy components. To reduce this effect, a two-dimensional median filter is applied to the mask before reconstruction:

$$M(f, t) \leftarrow \text{MedianFilter}_{F_f \times F_t}(M(f, t)),$$

where  $F_f$  and  $F_t$  are the filter extents in the frequency and time dimensions respectively.

5. **Signal reconstruction.** The smoothed mask is applied to the STFT:

$$\tilde{S}(f, t) = M(f, t) \cdot S(f, t)$$

The denoised signal is recovered via inverse STFT:

$$\tilde{y}(t) = \text{ISTFT}\{\tilde{S}(f, t)\}$$

**Relation to the Wiener filter.** The classical Wiener filter minimises the MSE between the estimated and true clean signal. Assuming signal and noise are uncorrelated and stationary, the optimal frequency-domain gain is:

$$H(f) = \frac{P_s(f)}{P_s(f) + P_n(f)},$$

where  $P_s(f)$  and  $P_n(f)$  are the signal and noise power spectral densities. Since both are unknown in practice, we substitute  $P_s(f) \approx |S(f, t)|^2$  and  $P_n(f) \approx N(f)^2$ , giving:

$$H(f) \approx \frac{\text{SNR}^2}{\text{SNR}^2 + 1}.$$

Our mask shares the same structural form – a ratio of signal strength to signal-plus-noise strength – but operates in the amplitude rather than the power domain:

$$M(f, t) = \frac{\text{SNR}}{\text{SNR} + \alpha} \quad \text{vs} \quad H(f) \approx \frac{\text{SNR}^2}{\text{SNR}^2 + 1}.$$

With  $\alpha = 1$ ,  $M(f, t) = H(f)$  at  $\text{SNR} = 1$ . Below this point,  $M > H$ : our mask is less aggressive than the Wiener gain, preserving faint signal components that would otherwise be suppressed. Above it,  $M < H$ , but bins with high SNR are dominated by signal regardless, so the additional attenuation should have little consequence.

Moreover, the Wiener stationarity assumption is violated in ecological audio: species vocalizations are sparse and intermittent, while background noise fluctuates between microphone hiss, wind, and overlapping species.

Figure 13 illustrates the effect of soft spectral gating on randomly sampled audio recordings from each `class_name`. For each example, the Mel spectrogram before and after noise reduction is shown. The Mel spectrograms highlight a clear reduction in background noise while preserving the dominant acoustic patterns of the target species. This observation was also confirmed through manual listening of the processed audio samples.

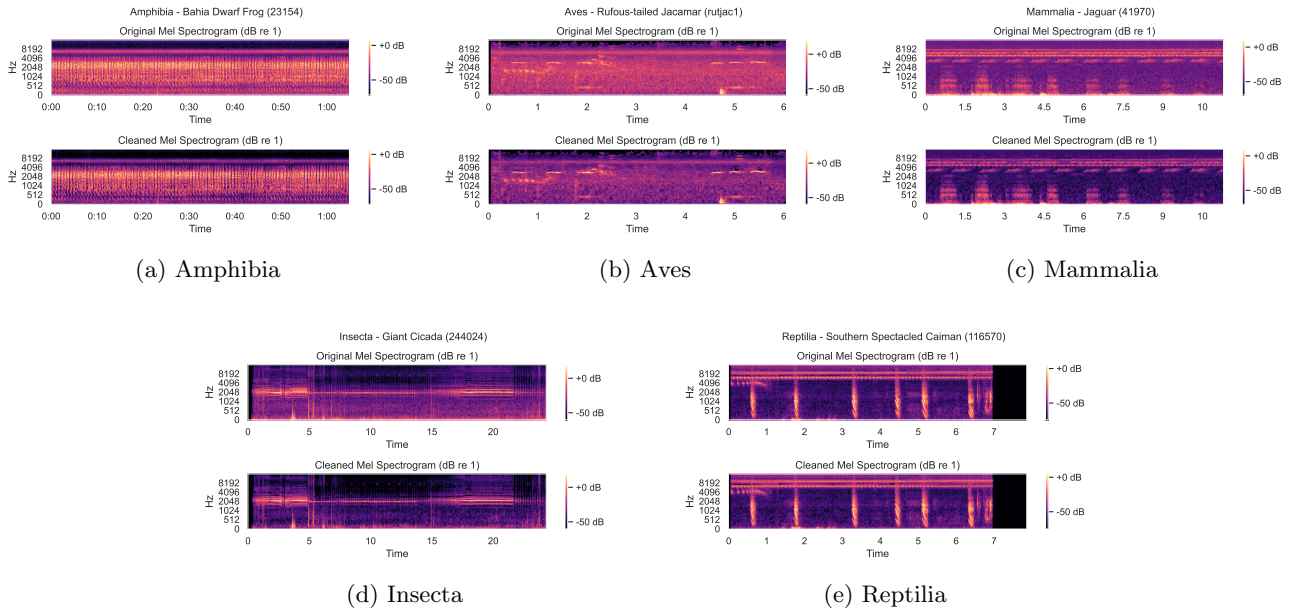


Figure 13: Examples of soft spectral gating applied to clean audio recordings across different taxa. For each sample, the Mel spectrogram before and after noise reduction is displayed

Figure 14 shows the effect of noise reduction on two randomly sampled soundscape recordings. Similar to the clean audio examples, the target species vocalizations become more distinguishable after spectral gating, which is also reflected in the corresponding Mel spectrograms.

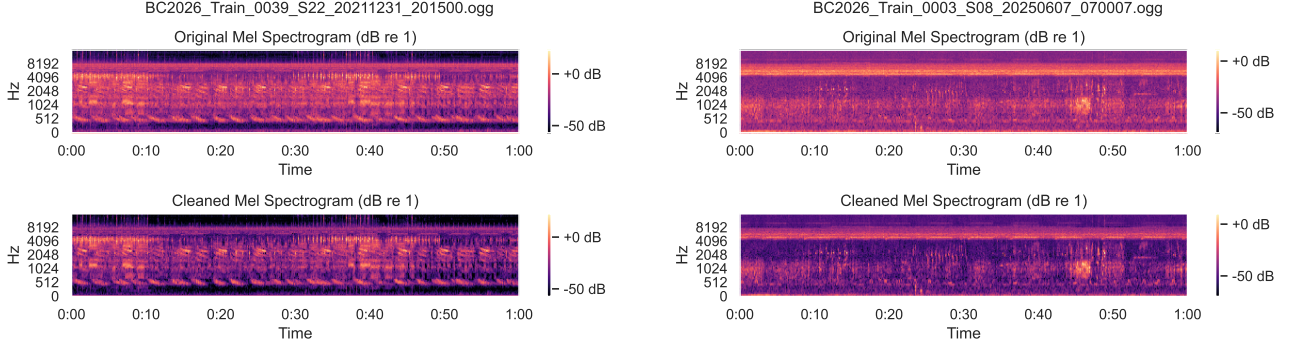


Figure 14: Examples of soft spectral gating applied to soundscape recordings, showing Mel spectrograms before and after noise reduction

### 6.3 Clean audio silence trimming

For silence trimming, an audio processing library `librosa` uses a threshold in dB relative to a reference level to determine silent segments. We use the mean of the signal as the global reference instead of the maximum, since the latter can be overly sensitive to isolated loud events (e.g., microphone pops or accidental contact) while the rest of the recording remains relatively quiet.

We set the threshold to 20 dB below the reference level, meaning that any frame with energy below 10% of the reference is considered silence. This follows directly from the definition of decibels:

$$D[k] = 20 \log_{10} \left( \frac{E[k]}{\text{ref}} \right)$$

$$D[k] < -20 \text{ dB} \Rightarrow \text{silent frame}$$

$$\frac{E[k]}{\text{ref}} < 10^{-1}$$

$$E[k] < 0.1 \cdot \text{ref}$$

$E[k]$  is the RMS energy of frame  $k$   
 $D[k]$  is the energy in decibels (dB)  
 $k$  is the frame index

Figure 15 shows examples of the trimming effect, including the shortest resulting recordings (subfigure 15a) and those with the largest duration reductions (subfigure 15b). For most recordings, the trimming effect is small: in 95% of cases, the duration change is below 1.6 seconds, indicating that the procedure removes silence rather than meaningful signal.

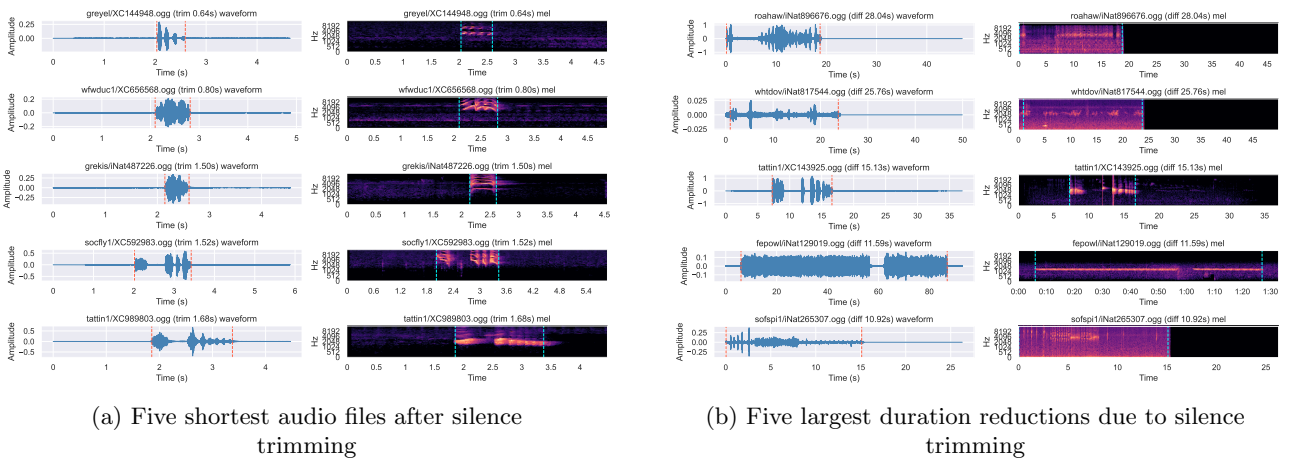


Figure 15: Effect of silence trimming on audio duration

## 7 Feature engineering

In this section, we discuss how features were extracted from audio to capture species-specific characteristics, how these features were aggregated, and how the audio was segmented into chunks.

Since soft spectral gating reduces noise by attenuating low-SNR components, we prioritize features that describe the underlying time–frequency structure of the signal, rather than descriptors that are more sensitive to noise-like properties, such as zero-crossing rate and spectral flatness.

Moreover, in this section we present each feature computed on five randomly sampled audio recordings per `class_name`, in order to better understand what each feature measures and how they differ across taxonomic classes. The same set of audio recordings is used across all feature groups to ensure a consistent comparison.

### 7.1 Audio segmentation and temporal aggregation

Most features presented in this section are computed at the time-frame level. To avoid excessive dimensionality (e.g., when flattening all frames), they are aggregated along the temporal axis using summary statistics. In particular, we compute the mean, standard deviation, and the 10th and 90th percentiles, which are preferred over the minimum and maximum to reduce sensitivity to outliers.

Our model variants, presented in Section 5.4, split audio into fixed-length chunks of size  $d_c$ . Unlike soundscape recordings, clean audio durations are not normalized, which requires handling incomplete segments at the end of each recording. To avoid generating many short segments or introducing artificial padding (e.g., with zeros or boundary values), we use strict  $d_c$ -second segmentation without overlap constraints. Figure 16 illustrates an example with  $d_c = 5$  seconds and zero overlap.

For recordings shorter than 5 seconds, we avoid padding and instead compute features using the available signal only, since padding would introduce artificial information and bias the resulting feature statistics.

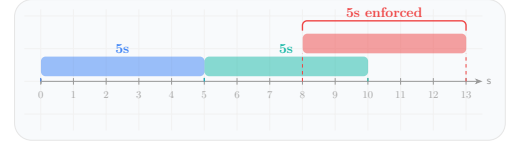


Figure 16: Audio segmentation into 5-second chunks with enforced last window (example)

### 7.2 Mel-spectrograms

Common time–frequency representations for audio include linear spectrograms, Mel-spectrograms, and MFCCs (Figure 6). In this project, we retained Mel-spectrograms with 128 Mel bands as the main representation, as they provide a suitable compromise between dimensionality reduction and preservation of relevant acoustic information.

Linear spectrograms are the lowest compression technique, it keeps the same frequency resolution over the whole spectrum, with  $n_{\text{fft}} = 1024$  they would produce 513 bins. In contrast, Mel-spectrograms allow to compress frequencies according to the Mel-scale, where frequency channel range is narrower at low frequencies and wider at high frequencies, which reflects how humans tend to perceive difference at different frequencies.

As for MFCC, they are the highest compression technique, but they are preferred in speech recognition, where the assumption of Discrete Cosine Transformation (DCT) holds, the fact that neighboring frequency channels look similar (energy-wise) to each other and this should be true everywhere along the frequency axis. However, as we have seen in spectrograms species sounds, especially for *Mammalia* or *Aves*, vocalizations are sparse, and hence this assumption is broken.

$$S_{\text{dB}}(m, t) = 10 \log_{10} \left( \frac{S(m, t)}{S_{\text{max}}} \right)$$

$$S_{\text{max}} = \max_{m, t} S(m, t)$$

$m$  denotes the Mel frequency bin

$t$  denotes the time frame

### 7.3 Spectral dynamics

To capture temporal dynamics, i.e. how spectral energy evolves over short time scales, we compute delta and delta-delta features.

The delta feature measures the first-order temporal variation of the spectrum by comparing neighbouring frames within a window of size  $N$ :

$$\frac{\partial S}{\partial t}(f, t) = \frac{\sum_{n=1}^N n(S(f, t+n) - S(f, t-n))}{2 \sum_{n=1}^N n^2},$$

where the weighting by  $n$  assigns greater importance to frames farther from  $t$ , resulting in a noise-resistant estimate of the local slope.

A positive value of  $\frac{\partial S}{\partial t}$  at frequency bin  $f$  indicates increasing energy at that frequency, corresponding to call onsets or upward frequency sweeps, while a negative value indicates decreasing energy, as in call offsets or downward sweeps. To distinguish between smooth evolution and complex temporal structure such as bursts or modulated patterns (e.g. trills or warbles), we also compute the second-order delta-delta feature, which captures the acceleration of spectral changes:

$$\frac{\partial^2 S}{\partial t^2}(f, t) = \frac{\partial}{\partial t} \left( \frac{\partial S}{\partial t}(f, t) \right).$$

Figure 17 illustrates the Mel-spectrogram together with the corresponding delta and delta-delta representations for a sample audio recording from each taxonomic class.

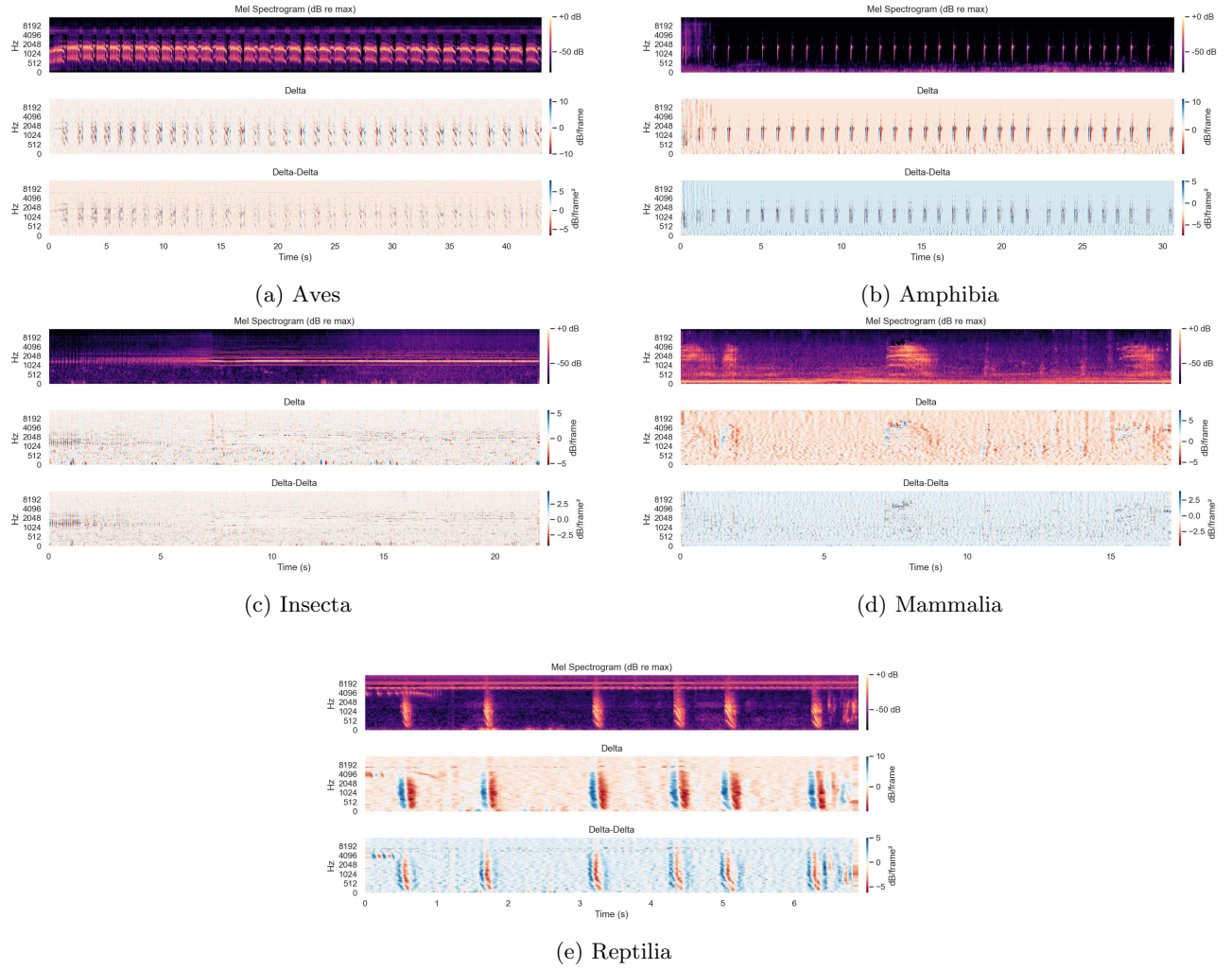


Figure 17: Mel-spectrograms and associated spectral dynamics feature representations across representative samples from different taxonomic classes.

## 7.4 Spectral shape and temporal activity

To characterize each frame’s acoustic content – its brightness, spectral spread, tonality, noisiness, and temporal dynamics – we compute the following spectral features: centroid, bandwidth, rolloff, contrast, entropy, flux, an energy proxy, a burstiness index, and inter-event gaps.

### 7.4.1 Frequency-related features

The **spectral centroid** corresponds to the energy-weighted mean frequency:

$$\text{Centroid}(t) = \frac{\sum_m f_m S(m, t)}{\sum_m S(m, t)}$$

$f_m$  denotes the frequency associated with Mel bin  $m$

Higher centroid values indicate that spectral energy is concentrated at higher frequencies.

The **spectral bandwidth** measures the spread of the spectrum around the centroid:

$$\text{Bandwidth}(t) = \sqrt{\frac{\sum_m S(m, t) (f_m - c_t)^2}{\sum_m S(m, t)}}$$

$c_t$  is the spectral centroid

A narrow bandwidth indicates that energy is concentrated within a limited frequency range (tonal sounds), whereas a wide bandwidth indicates a more dispersed spectral distribution (noisy or broadband sounds).

The **spectral rolloff** is defined as the frequency  $f_r$  such that:

$$\sum_{f \leq f_r} S(f, t) = \alpha \sum_f S(f, t)$$

$\alpha$  denotes the rolloff proportion

Given a threshold  $\alpha$ , low rolloff frequencies indicate that  $\alpha$  of the spectral energy is contained in lower frequency bands, while higher rolloff values indicate that this energy threshold is reached at higher frequencies.

### 7.4.2 Temporal-energy dynamics

The **spectral energy proxy** is related to the classical root-mean-square (RMS) energy, which is defined in the time domain and commonly used as an indicator of loudness and overall signal intensity per time frame. Instead of computing energy directly in the time domain, we operate in the frequency (Mel) domain by averaging the Mel-spectrogram power:

$$E(t) = \frac{1}{M} \sum_m S(m, t)$$

$M$  is the number of Mel bins

This formulation provides an approximation of RMS energy while allowing us to rely solely on precomputed Mel-spectrograms, avoiding the need to return to the raw waveform. This approximation is justified by *Parseval’s theorem*, which states that signal energy is preserved (up to a scaling factor) between the time and frequency domains.

**Spectral flux** measures how much the spectrum changes between consecutive frames (the L2 distance between spectral representations of successive time frames):

$$\text{Flux}(t) = \sqrt{\sum_m (S(m, t) - S(m, t-1))^2}$$

$$\text{Flux}(0) = 0$$

Flux helps distinguish between static or slowly varying audio and audio with sharp temporal transitions.

**Burstiness** captures the presence of intermittent packets of activity, in contrast to spectral flux, which cannot distinguish whether activity is temporally clustered or uniformly distributed. For example, both continuously

high-energy and continuously low-energy signals may exhibit low flux. Burstiness is defined per time frame using local moving windows:

$$B(t) = \frac{\sigma_t - \mu_t}{\sigma_t + \mu_t}$$

$\mu_t$  is the moving average

$\sigma_t$  is the moving standard deviation

High burstiness indicates sparse, impulsive events, while low burstiness indicates stable or continuous background activity, reflecting the intermittency of events.

**Spectral entropy** measures the uniformity of the spectral energy distribution within a frame. First, the Mel spectrum is normalized into a probability distribution:

$$p(m, t) = \frac{S(m, t)}{\sum_m S(m, t)}$$

Spectral entropy is then defined as:

$$H(t) = - \sum_m p(m, t) \log p(m, t)$$

Low entropy indicates concentration of energy in a few frequency bins (tonal or structured sounds), while high entropy indicates that energy is spread across many bins (noisy or unstructured sounds).

#### 7.4.3 Event structure

**Peaks**, or **events**, are detected using an adaptive global threshold computed separately for each audio recording:

$$\theta = \mu_E + k\sigma_E$$

$\mu_E$  is the mean spectral energy proxy

$\sigma_E$  is the standard deviation of the spectral energy proxy

$k$  is a scaling coefficient; larger values of  $k$  lead to detection of only strong bursts

For each frame  $t$ , distances to neighboring peaks (**inter-event gaps**) are computed:

$$d_{\text{prev}}(t) = t - t_{\text{prev peak}}$$

$$d_{\text{next}}(t) = t_{\text{next peak}} - t$$

This formulation allows us to capture both regular, periodic event structures and irregular, sparse bursts of activity.

#### 7.4.4 Spectral Contrast

Spectral contrast measures how structured the frequency content is within each time frame by comparing strong and weak components of the spectrum across frequency subbands. Instead of analysing the entire spectrum globally, the signal is divided into subbands, which provides more stable and locally meaningful statistics than operating directly on individual Mel bins.

Within each subband, spectral contrast is defined as:

$$\text{Contrast} = \log \left( \frac{\text{Peak Energy}}{\text{Valley Energy}} \right)$$

Thus, spectral contrast quantifies how strongly dominant spectral components stand out relative to the surrounding spectral floor. High spectral contrast indicates well-defined harmonic or tonal structure, whereas low contrast corresponds to noise-like or spectrally uniform signals.

Figure 17 illustrates the Mel-spectrogram along with the associated spectral-shape and temporal dynamics features for a sample audio recording from each taxonomic class.

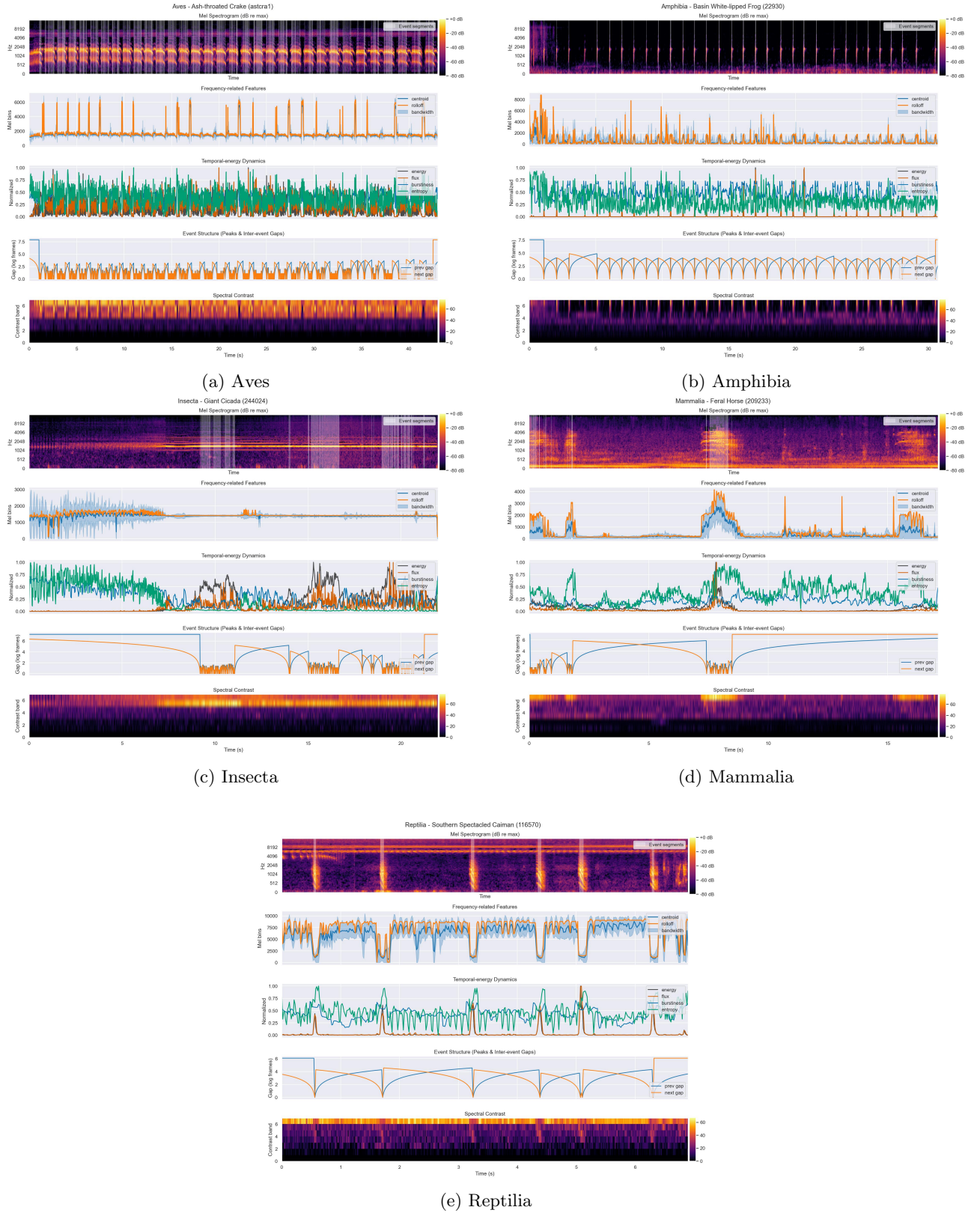


Figure 18: Mel-spectrograms and associated spectral and temporal feature representations across representative samples from different taxonomic classes.

## 7.5 Texture analysis

For texture analysis of the Mel-spectrogram, we adopt two descriptors from image processing: the Gray-Level Co-occurrence Matrix (GLCM) and Local Binary Patterns (LBP). In this formulation, the Mel-spectrogram is treated as a two-dimensional image defined over time and frequency, where pixel intensity represents energy.

Since both methods require discrete inputs, the spectrogram  $S(m, t)$  is first globally normalized using its minimum and maximum values:

$$\tilde{S}(m, t) = \frac{S(m, t) - S_{\min}}{S_{\max} - S_{\min}}$$

$$S_{\min} = \min_{m, t} S(m, t), \quad S_{\max} = \max_{m, t} S(m, t)$$

It is then quantized into  $L$  discrete levels:

$$Q(m, t) = \text{clip} \left( \left\lfloor \tilde{S}(m, t)(L - 1) \right\rfloor, 0, L - 1 \right) \in \{0, \dots, L - 1\}$$

To capture local spectro-temporal evolution, both GLCM and LBP are computed over sliding temporal windows centered at each time frame. For each time index  $t$ , we define a local patch:

$$Q_t = Q[:, \max(0, t - w) : \min(T, t + w)]$$

$w$  is the temporal context size (in frames)

### 7.5.1 Gray-Level Co-occurrence Matrix (GLCM)

The Gray-Level Co-occurrence Matrix (GLCM) characterizes texture by modeling the spatial co-occurrence of quantized intensity levels in the spectrogram. Unlike standard spectral or temporal descriptors that treat time and frequency independently, GLCM captures joint local structure by encoding how intensity levels co-occur in neighboring positions.

We compute a sequence of local GLCMs over sliding temporal windows, allowing the texture representation to evolve over time. This is particularly relevant for soundscapes, where acoustic activity is non-stationary and species are often present only briefly.

For each window  $Q_t$ , we construct two co-occurrence matrices corresponding to different adjacency directions. The **temporal GLCM** is defined using horizontal (time-direction) adjacency:

$$(Q_t(m, \tau), Q_t(m, \tau + 1))$$

The **frequency GLCM** is defined using vertical (frequency-direction) adjacency:

$$(Q_t(m, \tau), Q_t(m + 1, \tau))$$

These constructions yield matrices  $C_{\text{time}}^{(t)}(i, j)$  and  $C_{\text{freq}}^{(t)}(i, j)$ , which count how often intensity levels  $i$  and  $j$  co-occur locally in each direction. Each matrix is then normalized into a probability distribution:

$$P_{\text{time}}^{(t)}(i, j) = \frac{C_{\text{time}}^{(t)}(i, j)}{\sum_{a, b} C_{\text{time}}^{(t)}(a, b)}$$

$$P_{\text{freq}}^{(t)}(i, j) = \frac{C_{\text{freq}}^{(t)}(i, j)}{\sum_{a, b} C_{\text{freq}}^{(t)}(a, b)}$$

#### GLCM statistics

**Contrast** measures local intensity variation:

$$\text{Contrast}^{(t)} = \sum_{i, j} P^{(t)}(i, j)(i - j)^2$$

High values correspond to abrupt spectral transitions (e.g., transient vocal bursts), while low values indicate smooth harmonic structure.

**Homogeneity** measures local smoothness:

$$\text{Homogeneity}^{(t)} = \sum_{i,j} \frac{P^{(t)}(i,j)}{1 + |i - j|}$$

High values indicate stable and consistent textures, while low values correspond to irregular or noisy patterns.

**Energy** measures the concentration of co-occurrence structure:

$$\text{Energy}^{(t)} = \sum_{i,j} \left( P^{(t)}(i,j) \right)^2$$

High values indicate highly regular or repetitive patterns, while low values indicate heterogeneous textures.

**Entropy** measures uncertainty in local structure:

$$\text{Entropy}^{(t)} = - \sum_{i,j} P^{(t)}(i,j) \log P^{(t)}(i,j)$$

High values indicate complex and unpredictable acoustic scenes, while low values correspond to structured or periodic signals.

Figure 19 illustrates representative Mel-spectrograms together with their corresponding GLCM-based texture representations across taxonomic classes.

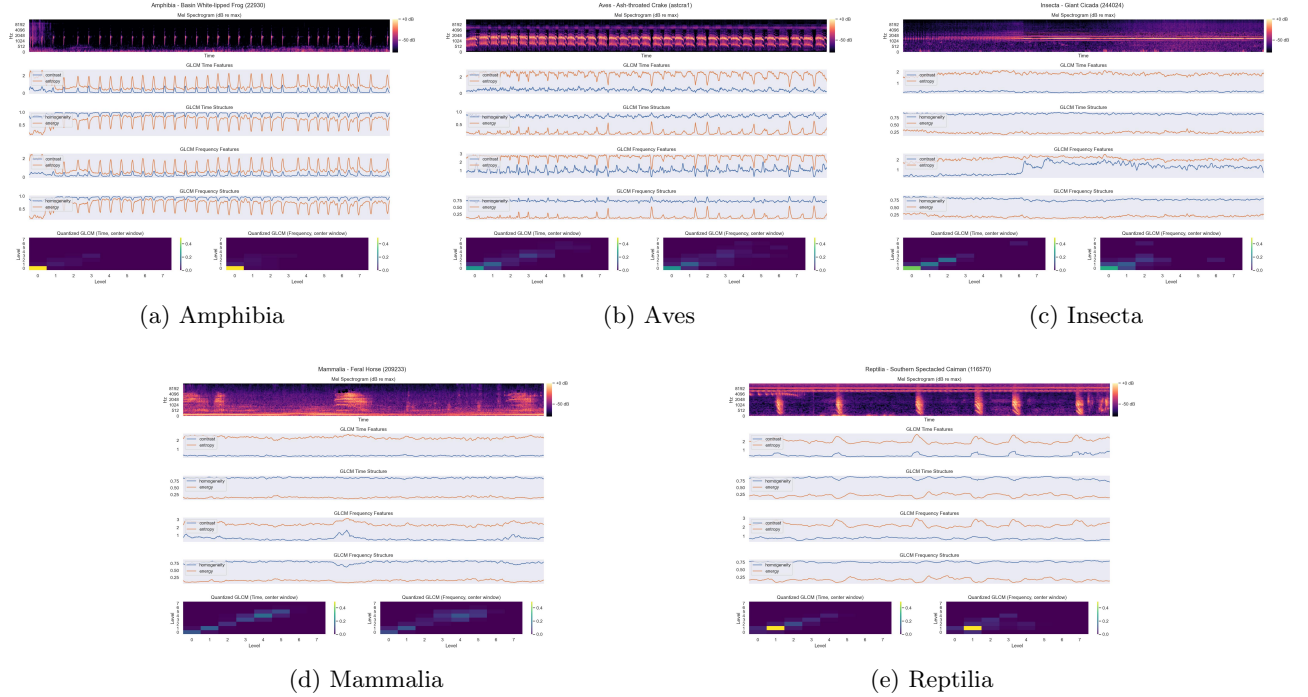


Figure 19: GLCM-based temporal and frequency texture representations across taxonomic classes.

### 7.5.2 Local Binary Patterns (LBP)

Local Binary Patterns (LBP) encode fine-grained spectro-temporal structure by comparing each quantized spectrogram value with its local neighborhood. Unlike GLCM, which models co-occurrence statistics, LBP explicitly encodes the local configuration of intensity variations.

For each central pixel  $x_c$ , neighboring pixels  $x_n$  are compared:

$$b_n = \begin{cases} 1 & \text{if } x_n \geq x_c \\ 0 & \text{otherwise} \end{cases}$$

The resulting binary pattern is encoded as:

$$\text{LBP} = \sum_{n=0}^7 b_n 2^n \in \llbracket 0, 255 \rrbracket$$

Values at the extremes have intuitive interpretations:

- 0: local peak (center dominates its neighborhood),
- 255: local valley (center is lower than all neighbors),
- intermediate values: directional or edge-like local structures.

For each time index  $t$ , LBP codes are computed over a local spectro-temporal window and summarized using a normalized histogram:

$$\hat{H}(k, t) = \frac{1}{Z_t} \sum_{(m, \tau) \in W_t} \mathbf{1}[\text{LBP}(m, \tau) = k]$$

$W_t$  denotes the local spectro-temporal window around time  $t$   
 $\in \llbracket 0, 255 \rrbracket$

$Z_t$  is a normalization constant ensuring a probability distribution

Compared to GLCM, LBP is more sensitive to localized directional and edge-like structures, such as harmonic ridges, chirp-like frequency sweeps, and sharp spectral discontinuities. It therefore complements GLCM by encoding fine-grained local geometry rather than only co-occurrence statistics.

Figure 20 illustrates LBP histogram-based texture representations across taxonomic classes, together with per-frame entropy, which reflects the diversity of local binary patterns within each temporal window. Higher entropy indicates that a wider variety of local structures is present, whereas lower entropy corresponds to more uniform or repetitive local configurations.

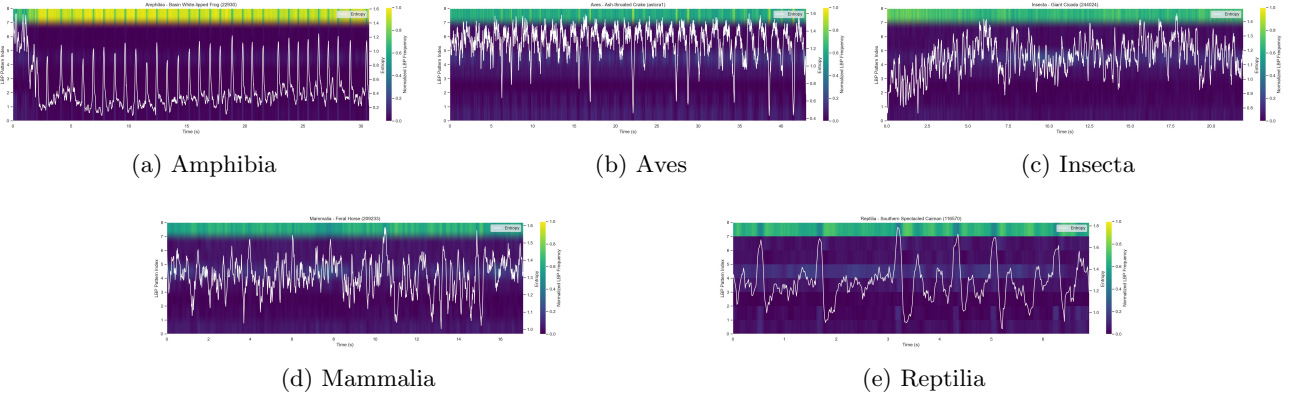


Figure 20: LBP histogram representations and associated per-frame entropy across taxonomic classes

## 7.6 Species Mel profiles

The inspiration for species profiles came from an initial inspection of Mel-spectrograms, which revealed that spectral energy tends to concentrate in different frequency regions depending on the species. To investigate this more systematically, we analyze the distribution of dominant Mel-frequencies, defined as the frequency bin with maximum energy at each time frame. These values are then aggregated into histograms.

We first compute these distributions at the taxonomic class level, and then repeat the analysis for up to five randomly sampled primary labels per class<sup>2</sup>. shows the resulting distributions across taxonomic classes, while Figure 22 presents KDE estimates for selected species within each class.

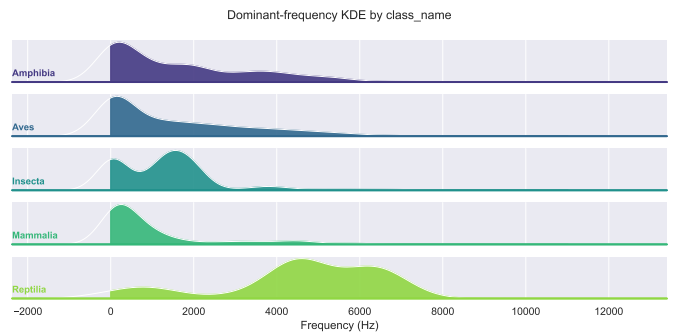


Figure 21: KDE of dominant Mel-frequency distributions across taxonomic classes

<sup>2</sup>For *Reptilia*, only one primary label is available, so no per-species plots are shown

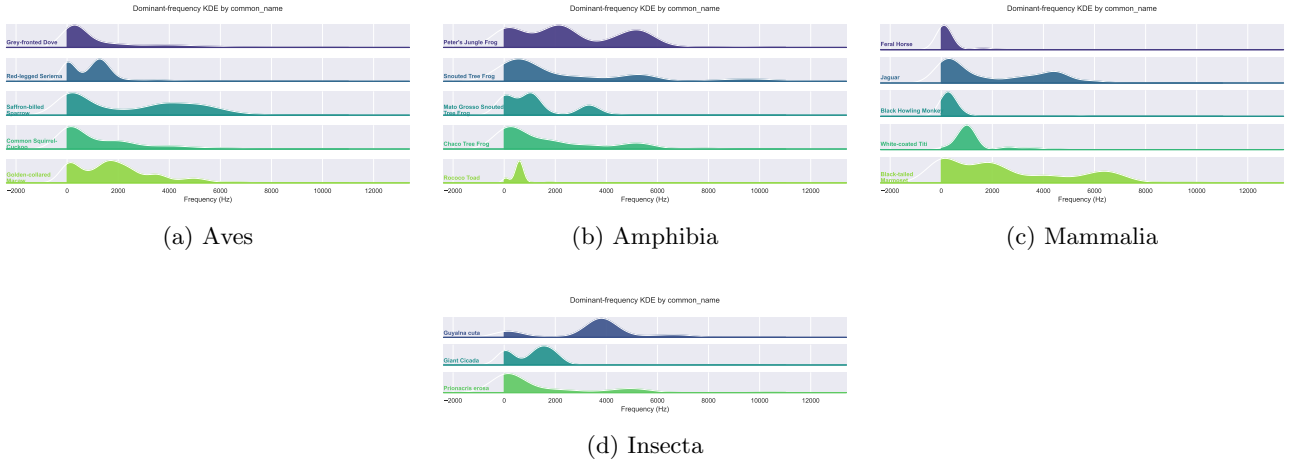


Figure 22: KDE of dominant Mel-frequency distributions for selected species within each taxonomic class

Although these estimates are imperfect due to time frame-level computation and the presence of silent or low-energy segments, they still suggest that different taxa operate at frequencies, with peaks occurring in different regions of the spectrum.

Building on this observation, we define a species profile as a global representation of spectral energy distribution across Mel-frequency bands for each `primary_label`. Profiles are computed over full audio recordings without segmenting them into fixed-length chunks. Figure 23 illustrates the pipeline used to construct a species profile for each primary label. Since multiple recordings are available per label, the final profile is obtained by aggregating individual profiles using a trimmed mean, which reduces the influence of outliers.

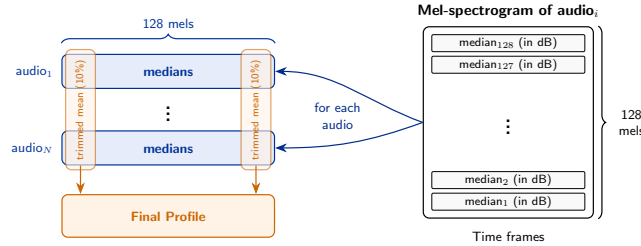


Figure 23: Pipeline for constructing species profiles from recordings of each primary label

Figure 24 shows the resulting profiles across taxonomic classes and selected species. Consistent with the KDE analysis, different species exhibit distinct patterns of energy concentration across Mel-frequency bands.

**Note.** Ideally, these profiles should have been computed after splitting the dataset into training and validation sets. However, to avoid heavily modifying the pipeline, we tolerate this small source of data leakage. Since the profiles are computed globally at the recording level rather than per time chunk, and because the training split represents around 80% of the dataset, this leakage should not significantly bias the evaluation.

### 7.6.1 Cosine similarity to species Mel profiles

To transform species profiles into features, we compute the cosine similarity between the Mel-spectrogram representation of an audio chunk and each species profile. Cosine similarity is particularly suitable in this setting because it measures similarity in the shape of spectral energy distributions while remaining invariant to overall magnitude. Magnitude-based similarity measures are less reliable here, since each Mel-spectrogram is normalized relative to its own maximum value, which varies across recordings and is therefore not directly comparable between samples.

$$\text{sim}(\mathbf{x}, \mathbf{p}^{(s)}) = \frac{\langle \hat{\mathbf{x}}, \hat{\mathbf{p}}^{(s)} \rangle}{\|\hat{\mathbf{x}}\| \cdot \|\hat{\mathbf{p}}^{(s)}\|}$$

$\mathbf{x}$  is the mean Mel-spectrogram representation of the chunk over  $M$  Mel bins

$\mathbf{p}^{(s)}$  is the Mel profile of species  $s$  over  $M$  Mel bins

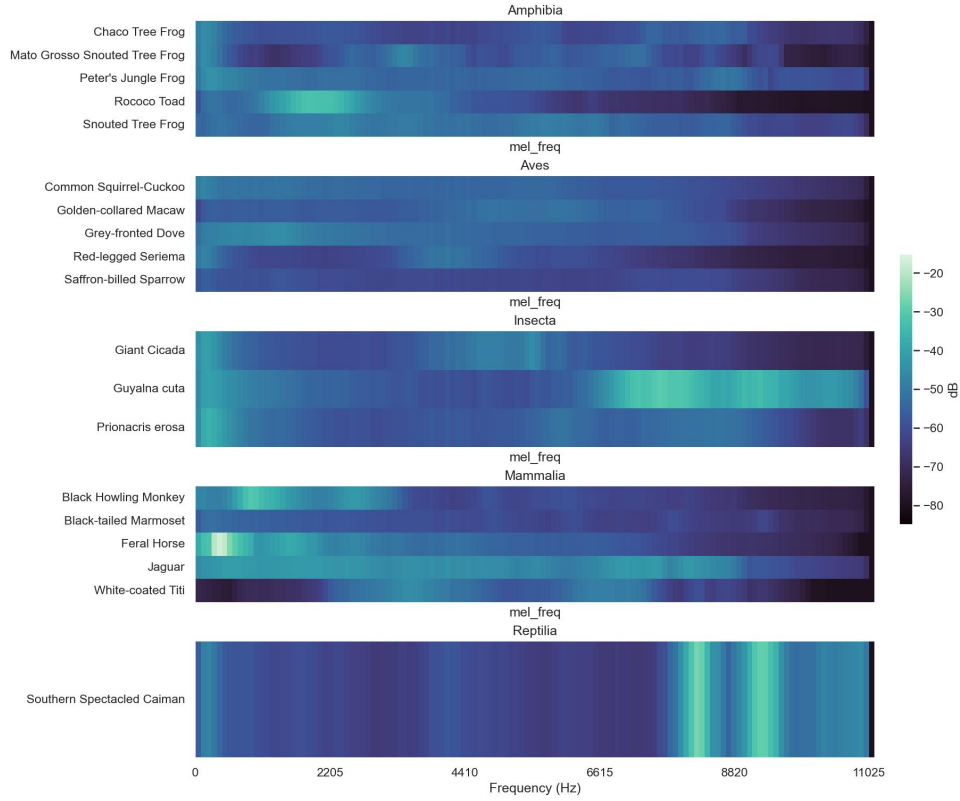


Figure 24: Heatmap of species profiles showing averaged Mel-frequency energy distributions across taxonomic classes and selected species

For illustration purposes, we randomly sampled one species and visualized how cosine similarity evolves across consecutive chunks. Figure 25 presents the evolution of profile similarity using 5-second chunk segmentation, while Figure 26 shows the same analysis using 1-second MIL segmentation.

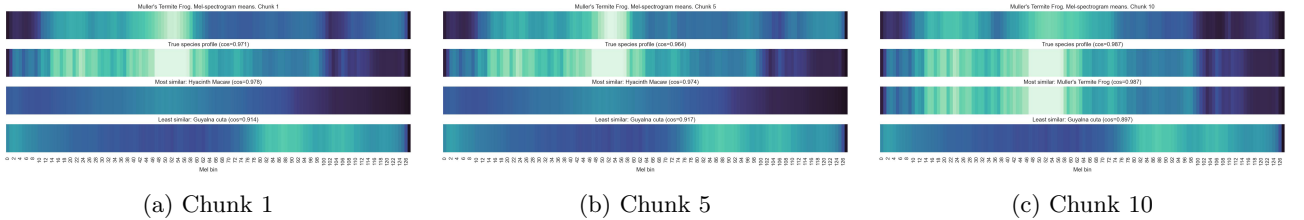


Figure 25: Evolution of cosine similarity between chunk-level Mel representations and species profiles for successive 5-second audio chunks.

## 7.7 Reducing the number of Mel bins

After aggregating frame-level statistics (mean, standard deviation, p10, and p90), the resulting feature vectors reached 1870 dimensions. However, many of the proposed features already capture complementary information related to energy variability (spectral entropy), temporal intermittency (burstiness and inter-event gaps), frame-to-frame spectral change (spectral flux), local texture structure (GLCM and LBP), and global spectral shape (species profiles). Consequently, retaining the full 128-bin Mel representation, together with its delta and delta-delta features, became unnecessarily redundant.

To reduce dimensionality, we compress the Mel representation from 128 to 32 bins, and apply the same reduction to delta and delta-delta features. Because each reduced bin aggregates a broader frequency region, we also omit the computation of per-Mel standard deviations.

Instead of recomputing lower-resolution Mel-spectrograms from scratch, we aggregate existing features by taking the median over groups of  $4 = \frac{128}{32}$  consecutive Mel bins for each statistic. This reduces the feature

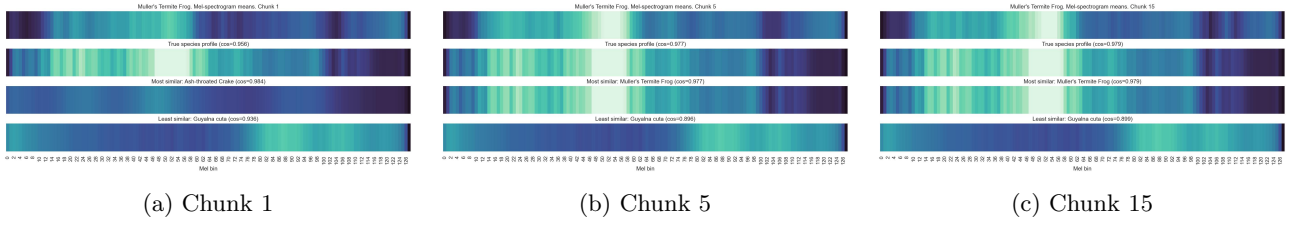


Figure 26: Evolution of cosine similarity between chunk-level Mel representations and species profiles for successive 1-second MIL chunks.

dimensionality from 1870 to 622, corresponding to approximately a threefold reduction.

## 7.8 Dimensionality reduction

To assess whether the proposed features produce meaningful joint representations, we apply UMAP to project the high-dimensional feature vectors into a two-dimensional embedding space. The goal is to examine whether recordings from the same species (**primary\_label**) cluster together, whether taxonomic structure emerges naturally, and whether soundscape segments align with clean recordings of the same species.

We perform this analysis at two temporal resolutions: 5-second chunks (Figures 27 and 28) and 1-second chunks (Figures 29 and 30). In both settings, the UMAP projection is learned using clean recordings only, while soundscape segments are projected into the same embedding space without retraining the manifold.

Figures 27a and 29a present the global embedding space, where samples are colored according to their taxonomic class. A clear large-scale organization of the feature space can be observed, with different taxonomic groups occupying distinct regions of the manifold.

Figures 27b– and 27f show the 5-second embeddings separately for each taxonomic class, with samples colored by **primary\_label**. The corresponding 1-second embeddings are shown in Figures 29b–29f. Within several taxonomic groups, recordings from the same species form localized clusters, particularly for Mammalia and Reptilia, suggesting that the extracted features capture species-specific acoustic structure.

Figures 28 and 30 illustrate the temporal trajectories of embeddings for four selected soundscapes. Each trajectory corresponds to the sequence of consecutive temporal windows projected into the learned UMAP space.

Overall, two main observations emerge. First, soundscape embeddings tend to form compact and locally coherent trajectories, indicating that the short-term acoustic content remains relatively stable within a recording. Second, a noticeable domain shift is often present between clean recordings and soundscapes. This effect is particularly visible in soundscape recording with a single species (Capuchin). In several cases, embeddings from clean recordings only partially overlap with those from soundscapes, likely due to background acoustic complexity, recording conditions, selection bias from curated clean recordings with high signal-to-noise target vocalizations, and the presence of potentially unannotated co-occurring species.

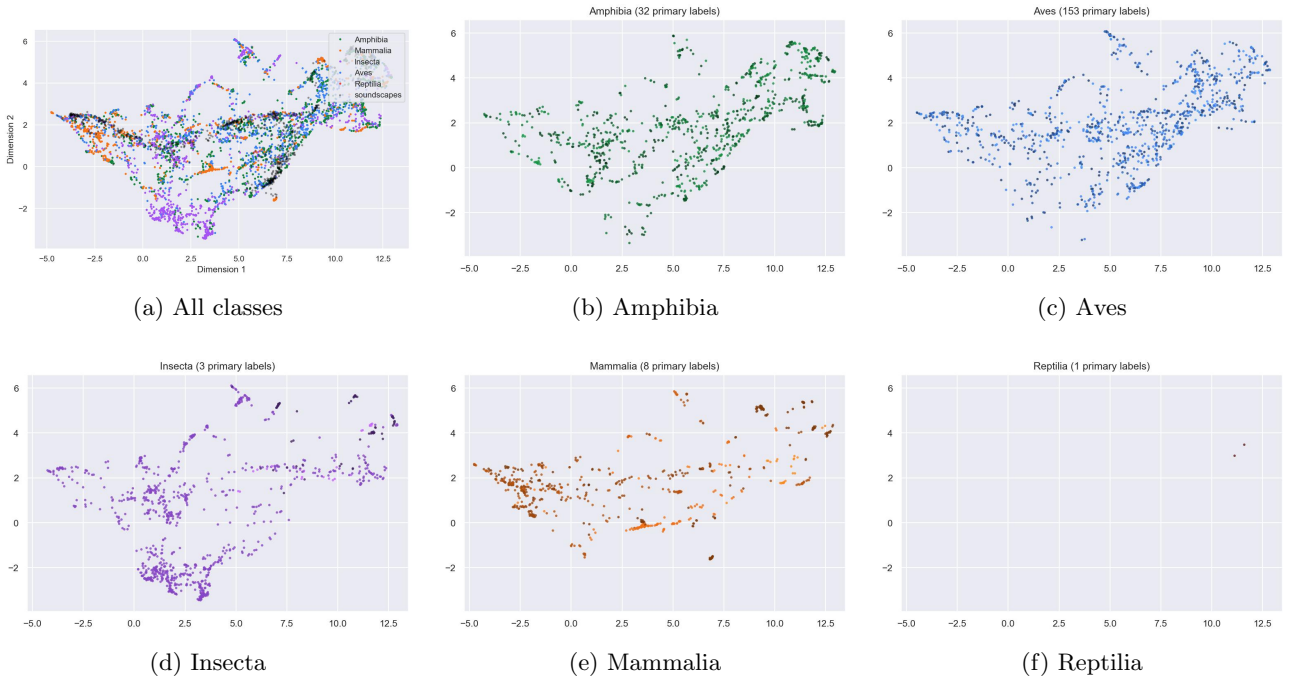


Figure 27: UMAP projection of 5-second audio embeddings. The top-left subplot shows the global embedding space colored by taxonomic class, while the remaining subplots display embeddings for each taxonomic class separately, colored by `primary_label`

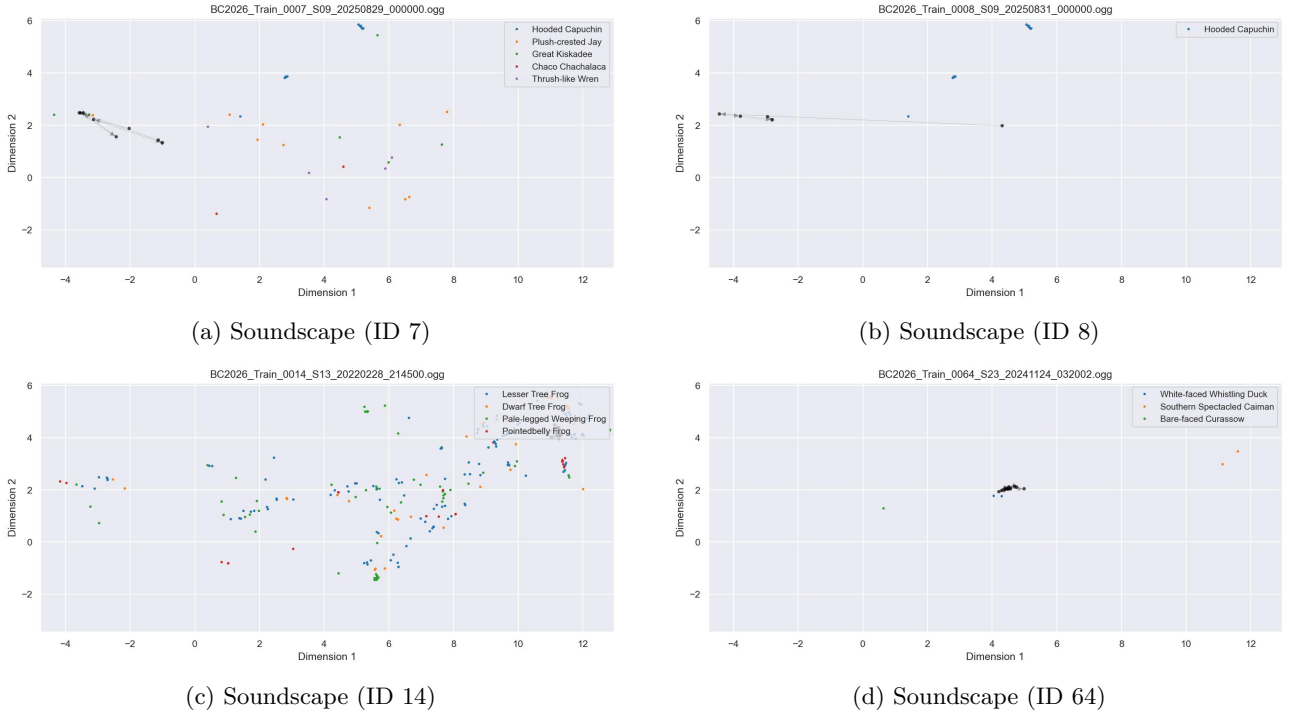


Figure 28: Temporal trajectories of soundscape embeddings projected into the UMAP space learned from clean 5-second recordings. Consecutive points correspond to adjacent temporal windows within the same recording

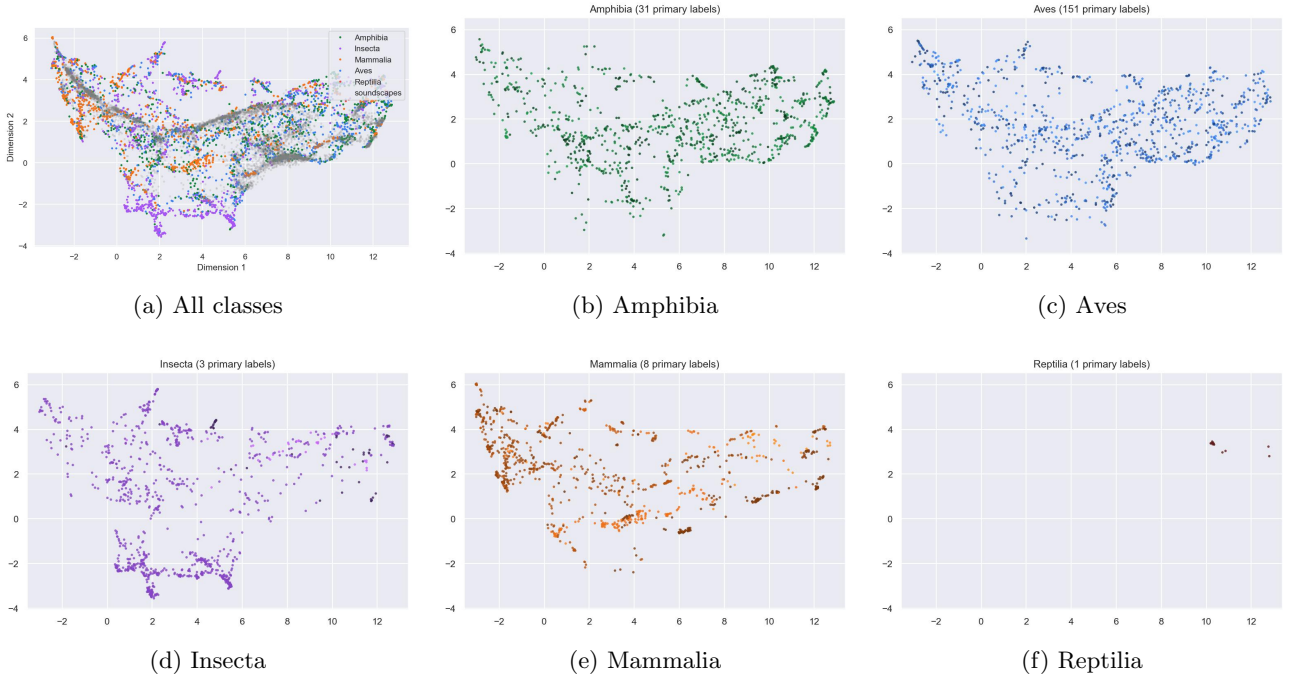


Figure 29: UMAP projection of 1-second audio embeddings. The top-left subplot shows the global embedding space colored by taxonomic class, while the remaining subplots display embeddings for each taxonomic class separately, colored by `primary_label`

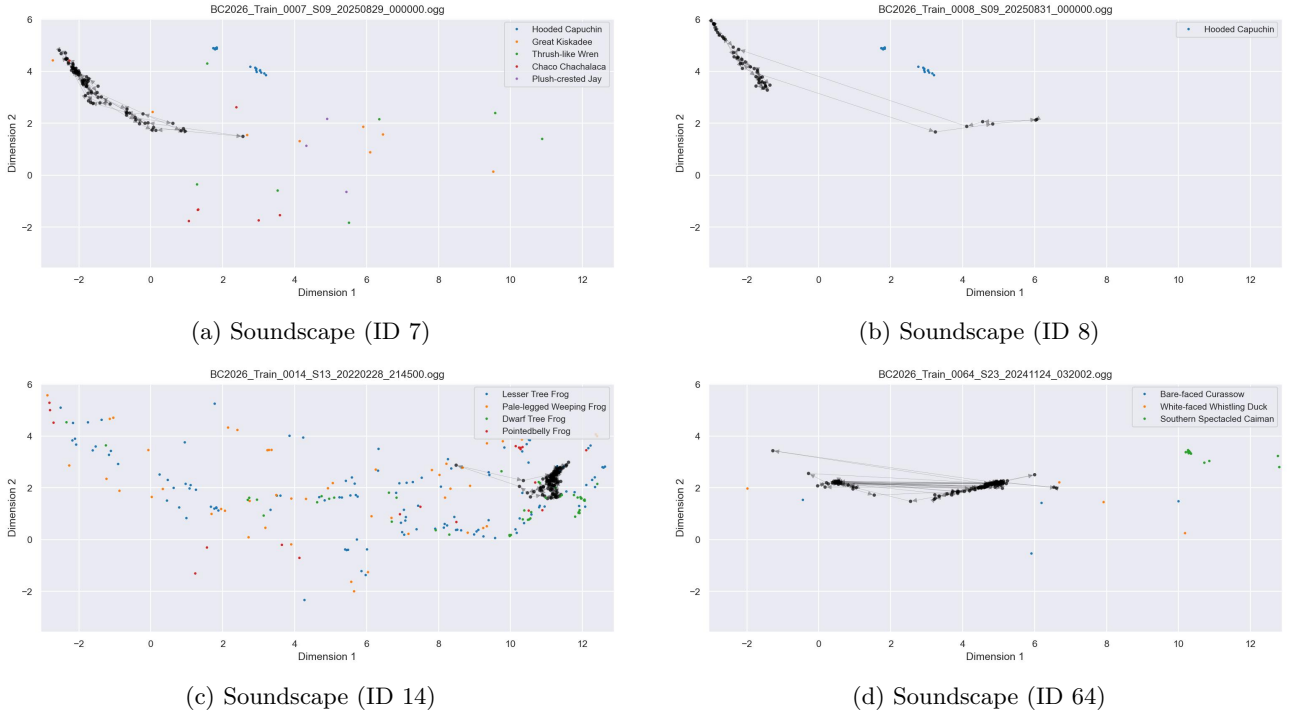


Figure 30: Temporal trajectories of soundscape embeddings projected into the UMAP space learned from clean 1-second recordings. Consecutive points correspond to adjacent temporal windows within the same recording

## 7.9 Soundscapes context features

As discussed in Section 5.5, we define additional context features for soundscape recordings in the second-stage model. As illustrated in Figure 10, several species appear to be strongly associated with specific recording sites (i.e., habitat locations), while others exhibit seasonal patterns, and some show diurnal or nocturnal activity. We refer to these variables as soundscape context features.

The recording site is encoded using one-hot encoding, since there are only 9 distinct sites. In contrast, month (period of 12) and hour (period of 24) are encoded using sinusoidal representations to preserve their periodic structure. Without this encoding, values such as December (12) and January (1) would be treated as far apart, despite being temporally adjacent.

$$\begin{aligned} &\cos\left(\frac{2\pi \cdot \text{hour}}{24}\right), \quad \sin\left(\frac{2\pi \cdot \text{hour}}{24}\right) \\ &\cos\left(\frac{2\pi \cdot \text{month}}{12}\right), \quad \sin\left(\frac{2\pi \cdot \text{month}}{12}\right) \end{aligned}$$

## 8 Experimental details

In this section we present how clean audio and soundscapes datasets are split into train and validation sets, discuss the metrics and provide the dimensionality of our dataset after clean audio segmentation into chunks.

### 8.1 Dataset dimensionality

Audio segmentation substantially increases the number of training samples by transforming full recordings into overlapping short-time representations. Table 5 presents the dataset size before and after audio segmentation.

| Dataset     | Full recordings | 5s chunks (2.5s overlap) | 1s chunks (0.5s overlap) |
|-------------|-----------------|--------------------------|--------------------------|
| Clean audio | 31,991          | 329,698                  | 1,680,578                |
| Soundscapes | 66              | 739                      | 6,651                    |

Table 5: Dataset size after temporal segmentation

### 8.2 Train/validation split

We perform grouped splitting at the recording (filename) level, followed by multilabel stratification. In the multilabel setting, exact stratification is generally not feasible because each sample can belong to multiple classes simultaneously. We therefore rely on the iterative multilabel stratification procedure implemented in the `iterstrat` library, which approximates balanced label distributions across splits.

This grouped strategy ensures that all chunks originating from the same recording are kept within the same subset, avoiding any overlap between training and validation sets. This is important because adjacent or overlapping segments from the same audio file can be highly similar, and splitting them across sets would introduce data leakage and lead to over-optimistic validation performance.

For clean audio, we use a single grouped train/validation split. Cross-validation and out-of-fold (OOF) evaluation are not applied due to the dataset size and the resulting computational cost of model training.

For soundscape data, which is considerably smaller, we instead use grouped multilabel cross-validation and out-of-fold predictions to obtain more stable and reliable evaluation results.

### 8.3 Evaluation metrics

Both the clean audio and soundscape datasets are highly imbalanced. Consequently, we prioritize macro-averaged metrics, which assign equal weight to all classes and prevent performance from being dominated by frequent species.

Since we rely on OvR classifiers, predictions for clean audio are obtained by selecting the class with the highest predicted probability.

For multi-label soundscapes, predictions are typically obtained by applying decision thresholds to class-wise probabilities. However, due to strong class imbalance, a single global threshold is often suboptimal. In practice, class-specific thresholds are usually required, with lower thresholds for rare species to improve recall and higher thresholds for frequent species. Due to time constraints, we did not perform threshold tuning. As a result, we focus on threshold-independent and ranking-based metrics, as well as top-k evaluation criteria.

### Clean single-label audios

1. **Accuracy**: proportion of correctly classified samples over the 206 classes, measuring overall classification performance.
2. **Top-5 Accuracy**: fraction of samples where the true class appears among the five highest-scoring predictions, useful in large label spaces with closely related species.
3. **Macro Precision / Recall / F1**: class-averaged metrics treating all species equally. Precision measures false positives, recall measures missed detections, and F1 combines both.
4. **Macro ROC AUC**: evaluates the ability of the model to rank positive samples above negatives independently of any threshold. It can be optimistic in highly imbalanced settings due to the dominance of true negatives.
5. **Label Ranking Average Precision (LRAP)**: evaluates the quality of the full ranking of classes per sample, reflecting how highly the true label is ranked among all candidates.
6. **Macro Average Precision (AP)**: measures ranking quality by assessing whether true positives receive higher scores than negatives across classes.

### Soundscapes (multi-label audios)

1. **Top-5 Accuracy**: measures whether at least one true species appears among the five highest-scoring predictions.
2. **Top-5 Precision**: fraction of the top-5 predicted species that are correct for a given recording.
3. **Top-5 Recall**: fraction of true species retrieved within the top-5 predictions.
4. **Macro ROC AUC**: primary Kaggle leaderboard metric, computed in a one-vs-rest manner per class.
5. **Label Ranking Average Precision (LRAP)**: evaluates how well all true species are ranked above irrelevant classes for each recording.
6. **Macro Average Precision (AP)**: measures ranking quality across all classes in the multi-label setting.

## 8.4 Model training

Given the large size of the dataset, we use mini-batch stochastic gradient descent via `SGDClassifier` (scikit-learn) with log loss, which allows us to recover probabilistic outputs. The training pipeline includes custom batching, class weighting, and early stopping. The full configuration of the `SGDClassifier` is provided in the appendix.

### 8.4.1 Batching and weighting

Since we adopt a OvR strategy, each classifier is reduced to a binary problem: positive samples correspond to the target class, while all other samples form the negative class. Because the dataset is highly imbalanced – and even more so after binarisation in the OvR setting – we address class imbalance through two mechanisms: batch construction and loss reweighting.

**Note.** We initially experimented with the standard inverse-frequency weighting used in scikit-learn,  $\frac{n}{C \cdot n_c}$ , together with the default `fit` function. However, due to the strong long-tail distribution, this led to excessively large weights, and the model appeared to learn little, as reflected by an almost random confusion matrix. We then removed loss weighting and relied only on perfectly balanced batches, i.e., each batch contained the same number of samples per class. This, however, introduced a bias towards *Amphibia*, despite it not being a majority

class. These observations motivated the design of a custom weighting scheme.

As a reminder (Section 5.3), we train both generalist classifiers over the full dataset to predict `class_name`, and specialist classifiers restricted to a single `class_name` to predict `primary_label`. In the latter case, weights are computed independently within each restricted dataset.

**Positive/negative loss weighting** Without reweighting, majority classes dominate the optimisation process, while minority classes risk being underrepresented. Let  $y \in \{1, \dots, C\}^n$  denote class labels over  $n$  samples. For each class  $c$ , we define unnormalised weights:

$$\tilde{w}_c^+ = \left(\frac{n}{n_c^+}\right)^\beta, \quad \tilde{w}_c^- = \left(\frac{n}{n_c^-}\right)^\beta, \\ n_c^+ \text{ is the number of samples in class } c, \quad n_c^- = n - n_c^+.$$

To improve numerical stability, we normalise these weights so that their mean equals one:

$$m_c = \frac{\tilde{w}_c^+ + \tilde{w}_c^-}{2}, \quad w_c^+ = \frac{\tilde{w}_c^+}{m_c}, \quad w_c^- = \frac{\tilde{w}_c^-}{m_c}.$$

**Sample-level weighting** To further stabilise optimisation and ensure that each batch contains a more balanced class distribution, we use weighted sampling with replacement. For each class  $c$ , we define sampling weights:

$$w_c^{\text{sample}} = \left(\frac{n}{n_c}\right)^{1-\beta}.$$

The probability of sampling class  $c$  is then:

$$p(c) \propto w_c^{\text{sample}}.$$

Under this construction, the expected contribution of class  $c$  during training satisfies:

$$\mathbb{E}[\text{contribution of class } c] \propto w_c^{\text{sample}} \cdot \tilde{w}_c^+ = \left(\frac{n}{n_c}\right)^{1-\beta} \left(\frac{n}{n_c}\right)^\beta \frac{1}{m_c} = \frac{n}{n_c \cdot m_c}.$$

With  $\frac{n}{n_c \cdot m_c}$  as the effective contribution weight for class  $c$ , the weighting scheme ensures that each positive class—the class that the model must distinguish from all others—contributes approximately equally to the training objective. The normalization factor  $m_c = \frac{\tilde{w}_c^+ + \tilde{w}_c^-}{2}$  increases when either  $\tilde{w}_c^+$  or  $\tilde{w}_c^-$  is large, which occurs in our dataset where *Aves* accounts for almost 90% of the samples. Consequently,  $m_c$  mitigates the excessively large inverse-frequency contributions induced by strong class imbalance. As a result, samples from majority classes are down-weighted, while samples from minority classes are up-weighted.

**Note.** For `primary_label` values not present in the training dataset, the predicted probability defaults to the empirical class prior estimated from class frequencies in the training set. For `primary_label` values present only in the soundscapes dataset and absent from the clean audio dataset, the predicted probability is set to 0, since no corresponding clean-audio training samples are available for the classifier.

## 8.5 Model architectures and training strategies

We investigate two training strategies and their corresponding model architectures. Both approaches use the same feature extraction pipeline but differ in their temporal segmentation strategy.

### 8.5.1 5-second chunk model

The first approach follows a more classical setup based on 5-second segmentation of clean audio recordings. Clean recordings are divided into 5-second chunks with a 2.5-second overlap, matching the annotation granularity of the soundscapes. Feature vectors are extracted at the chunk level, and OvR classifiers are trained directly on these representations.

At inference time on soundscapes, predictions are refined using a second-stage model that incorporates  $\mathbb{P}(\text{primary\_label})$ ,  $\mathbb{P}(\text{class\_name})$ ,  $\mathbb{P}(\text{primary\_label}) \cdot \mathbb{P}(\text{class\_name})$ , as well as additional soundscape context features, as illustrated in Figure 31. Since each `XGBClassifier` is trained in a one-vs-rest (OvR) setting, we simplify the feature space by providing only features relevant to the single `primary_label` it is meant to distinguish from all others. As a result, each `XGBClassifier` receives three probability-based features along with the context features. This

setup allows each classifier to focus on adapting probabilities for each `primary_label` separately. This design is not ideal, as it may ignore co-occurrence patterns between classes; however, the soundscapes dataset is very small and imbalanced, which limits the ability to reliably learn such co-occurrence patterns. We refer to this design as **feature space restriction**, where each classifier only receives features relevant to its target class in an OvR setting.

If a class is not present in the soundscapes training set, the final prediction defaults to  $\mathbb{P}(\text{primary\_label}) \cdot \mathbb{P}(\text{class\_name})$ .

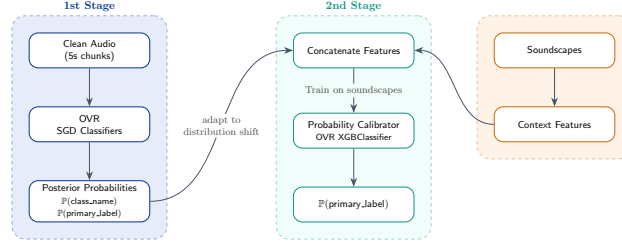


Figure 31: Architecture of the 5-second chunk model and adaptation for the soundscapes pipeline

### 8.5.2 MIL model

The second approach is inspired by Multiple Instance Learning (MIL). The idea to use an MIL-based approach came from manually listening to bird vocalizations in the soundscapes and observing that, although annotations are provided at the 5-second chunk level, individual bird calls often occur within much shorter time intervals. This creates a weakly supervised setting in which the exact temporal location of each species within a chunk is unknown.

To better capture this temporal structure, both clean recordings and soundscapes are segmented into 1-second windows with a 0.5-second overlap. These windows are then grouped into 5-second chunks, forming bags of nine instances per chunk.

In classical MIL, a model is trained at the instance level, and predictions are aggregated within each bag (e.g., max, mean, or log-sum-exp), with the loss computed at the bag level. To avoid significantly modifying our custom SGDClassifier training pipeline and to bypass the need to evaluate different aggregation choices (e.g., whether mean is too smooth or max is too noisy), we instead adopt a MIL-inspired two-stage approach:

1. A standard OvR classifier is first trained at the window level. This stage is not MIL-aware.
2. Window-level probabilities are then aggregated at the chunk level, and a second-stage OvR model is trained to refine and denoise the predictions.

Figure 32 illustrates the complete pipeline of the MIL model.

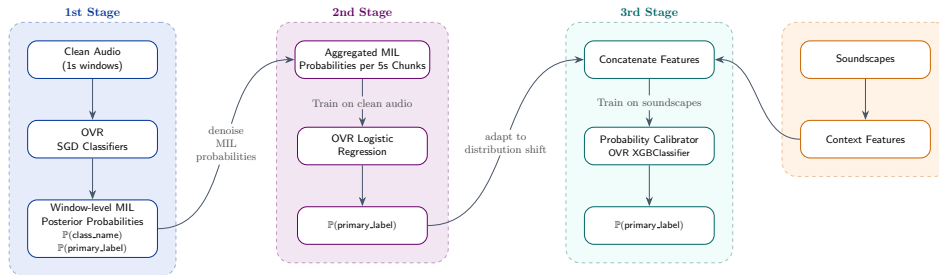


Figure 32: Architecture of the MIL model with window-level prediction, chunk-level aggregation, and adaptation for the soundscapes pipeline

### 8.5.3 Denoising probabilities

To denoise MIL probabilities from the first stage, we train a second-stage model, restricted to logistic regression due to the small number of input features and the simplicity of the training pipeline. It is trained in a

one-vs-rest (OvR) setting and takes as input aggregated probabilities for the `primary_label` and its corresponding taxonomic class, `class_name`, using a primary-label-specific feature space restriction. In this setup, each logistic regression model is responsible for denoising the probabilities associated with a single `primary_label`.

For each bag, we compute a set of statistical descriptors over the sequence of probabilities for both  $\mathbb{P}(\text{primary\_label})$  and  $\mathbb{P}(\text{class\_name})$ :

- Mean, to capture the overall activation, and standard deviation, to capture temporal stability.
- Maximum value and its position (argmax), to capture the strongest prediction and when it occurs.
- Minimum value and its position (argmin), to assess the lowest confidence and whether probabilities remain consistently above a given threshold.
- Proportion of values above the 80th percentile, to detect concentration and sustained strong activations.
- Mean absolute difference, to capture the smoothness of transitions between consecutive windows.

We also include interaction features between  $\mathbb{P}(\text{primary\_label})$  and  $\mathbb{P}(\text{class\_name})$

- Difference between their means, to capture overall disagreement.
- Correlation coefficient, to measure whether both probability sequences evolve in sync.
- Ratio  $\frac{\mathbb{P}(\text{primary\_label})}{\mathbb{P}(\text{class\_name})}$ , to quantify the relative confidence of the specialized model. A ratio of 1 indicates full agreement (identical probabilities), a value below 1 suggests that the class-level model is more confident than the specialist (possibly indicating another species within the same class), and a value above 1 may indicate a potential false positive, where the specialist is overly confident relative to the class-level prediction.

In total, the logistic regression model receives  $8 \times 2 + 3 = 19$  features.

#### 8.5.4 Adapting to soundscapes

Similarly to the `XGBClassifier` used in the 5-second chunk model (Section 8.5.1), the third-stage model takes as input  $\mathbb{P}_{\text{bag}}(\text{primary\_label})$  (as the best estimate obtained from clean audio), soundscape context features, as well as all aggregated probability features from the MIL pipeline, as described in Section 8.5.3, allowing the model to further adapt these probabilities to soundscapes. As in the case of the `XGBClassifier` in Section 8.5.1, we use a primary-label-specific feature restriction. The model outputs a calibrated estimate of  $\mathbb{P}_{\text{bag}}(\text{primary\_label})$ . For classes not present in the soundscapes training set, the predicted probability is kept as the output of the second-stage model, i.e.,  $\mathbb{P}_{\text{bag}}(\text{primary\_label})$ .

## 9 Evaluation

This section presents the evaluation metrics for both clean audio and soundscapes across the two model architectures, allowing for a clear comparison. For clean audio, normalized confusion matrices (normalized per true label and therefore to be read row-wise) are also provided. Confusion matrices are computed for all 234 species, even though only 206 species are present in the clean audio dataset. Finally, the soundscape evaluation includes selected audio examples to illustrate the predicted probabilities for each consecutive 5-second chunk.

### 9.1 Evaluation on clean audio

Table 6 presents evaluation metrics on clean audio for both the 5-second chunk model (before and after hierarchical fusion (HF) of probabilities  $\mathbb{P}(\text{primary\_label})$  and  $\mathbb{P}(\text{class\_name})$ ), and the MIL model.

| Model                          | Acc.          | Top-5 Acc.    | Macro Prec.   | Macro Rec.    | Macro F1      | Macro ROC-AUC | LRAP          | Macro AP      |
|--------------------------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| 5-second chunk model (no HF)   | 0.0031        | 0.0181        | 0.0015        | 0.0234        | 0.0026        | 0.8362        | 0.0557        | 0.1103        |
| 5-second chunk model (with HF) | 0.1289        | 0.2666        | <b>0.1967</b> | 0.1526        | 0.1117        | 0.7333        | 0.2088        | 0.0782        |
| MIL model (two-stage)          | <b>0.2042</b> | <b>0.4279</b> | 0.1845        | <b>0.1981</b> | <b>0.1616</b> | <b>0.8445</b> | <b>0.3162</b> | <b>0.1106</b> |

Table 6: Clean audio performance comparison across the 5-second chunk model and the MIL model

Figure 33 illustrates the confusion matrices for `class_name`, for `primary_label` before and after hierarchical fusion of probabilities. Figure 34 illustrates the confusion matrix of the MIL-inspired model after denoising and consequently aggregating probabilities per 5-second chunks. Figure 33 illustrates the confusion matrices for `class_name` and `primary_label`, before and after hierarchical fusion of probabilities, respectively<sup>3</sup>. Figure 34 shows the confusion matrix for the MIL model after denoising and aggregation of probabilities at the 5-second chunk level.

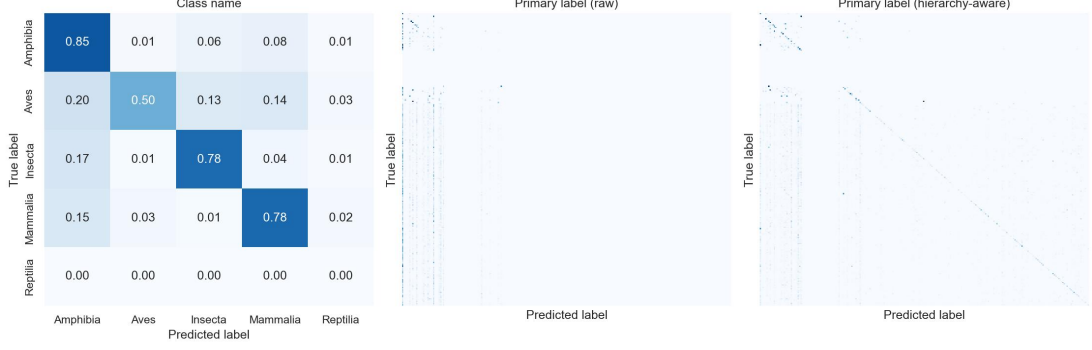


Figure 33: 5-second chunk model: confusion matrices before and after hierarchical probability fusion

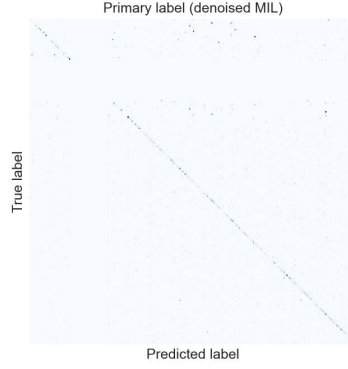


Figure 34: MIL model: confusion matrix after window-level denoising and chunk-level aggregation

Figure 35 presents per-class confusion matrices for each model variant, providing a more detailed view of which species are most challenging for the models and where most confusions occur.

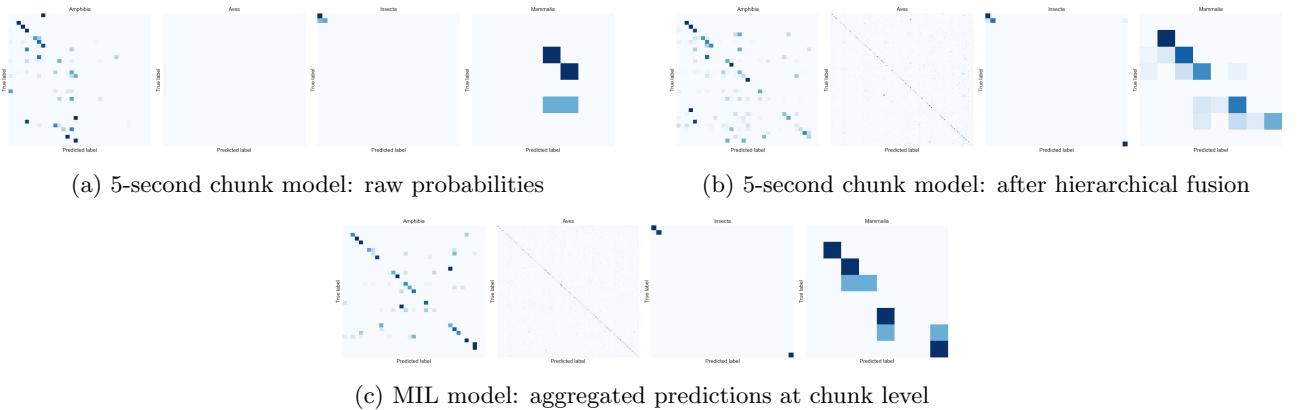


Figure 35: Per-class confusion matrices comparing the 5-second chunk model (raw and fused predictions) and the MIL model

For the 5-second chunk model architecture, hierarchical fusion leads to improved performance across all metrics except ROC-AUC and Average Precision. This suggests that the multiplication of probabilities slightly distorts

<sup>3</sup>*Reptilia* class is not represented since this class contains only one file and is therefore included in the training set only, not in validation

the ranking between positive and negative classes. However, despite this effect, the confusion matrices are significantly improved, with a clear diagonal structure across most taxonomic classes.

The MIL model consistently achieves the best performance across all metrics, with macro-precision remaining only slightly lower than the best 5-second chunk variant. This indicates that the MIL-inspired formulation is more effective at handling weak labels and the inherent sparsity of species vocalizations in soundscapes. The two-stage pipeline appears to provide a good approximation of a classical MIL framework while maintaining compatibility with the existing training setup.

## 9.2 Evaluation on soundscapes

Table 7 reports performance on soundscapes for the 5-second chunk model (with and without the second-stage refinement) and the MIL model. The first variant relies only on hierarchical fusion of probabilities, while the second incorporates an additional calibration stage for soundscape adaptation, and the third corresponds to the full three-stage MIL-inspired pipeline.

Introducing the second-stage model improves ranking-based metrics (ROC-AUC, Average Precision, and LRAP), indicating that true species are generally assigned higher scores than negative ones. However, Top-5 accuracy decreases by approximately 0.09, suggesting that although overall ranking improves, the correct species appear less frequently among the highest predicted labels, which is a limitation in practical detection scenarios.

The MIL model achieves the best overall performance across most metrics, particularly in Top-5 accuracy, ROC-AUC, and LRAP. Its Macro Average Precision remains comparable to the strongest 5-second chunk variant, indicating similar margin between scores assigned to true and non-true classes.

| Model                                     | Top-5 Acc.    | Top-5 Prec.   | Top-5 Rec.    | ROC-AUC       | mAP           | LRAP          |
|-------------------------------------------|---------------|---------------|---------------|---------------|---------------|---------------|
| 5-second chunk model (1-stage + HF)       | 0.5074        | 0.1494        | 0.1768        | 0.6253        | 0.1329        | 0.1248        |
| 5-second chunk model (2-stage refinement) | 0.4114        | 0.1323        | 0.1566        | 0.6886        | <b>0.2915</b> | 0.2570        |
| MIL model (three-stage pipeline)          | <b>0.6725</b> | <b>0.2936</b> | <b>0.3475</b> | <b>0.7487</b> | 0.2883        | <b>0.3303</b> |

Table 7: Soundscape performance comparison across the 5-second chunk model and the MIL model

To provide a qualitative evaluation of the soundscapes, we select three representative samples and plot the top-5 predicted classes with their associated probabilities. Green bars indicate correct predictions (true label within top-k), red bars indicate missed detections (true label not in top-k), and blue bars indicate false positives (predicted within top-k but not the true label). Each red bar is annotated with a rank indicator (starting from #1), which reflects how far the true class is from entering the top predictions.

Figures 36a–36c illustrate a case where the second-stage model is able to substantially increase the probability of the correct class, eventually ranking it at the top for the second annotated chunk. In contrast, the MIL model consistently places the correct class within the top-5 predictions for both chunks.

Figures 38 show a fully annotated soundscape dominated by amphibian vocalizations, where the 5-second chunk model struggles to consistently rank the correct classes at the top. The second-stage model improves the probabilities of relevant classes, but also activates multiple other amphibian classes with high confidence, suggesting confusion between acoustically similar species. The MIL model performs strongly in this scenario, placing the correct classes within the top-5 predictions for most chunks. This suggests that, in this case, amphibian species exhibit high acoustic similarity that is not fully captured by the features used in the 5-second chunk pipeline.

Finally, Figure 39 shows a soundscape where the 5-second chunk model with second-stage refinement outperforms the MIL model, both in Top-5 accuracy and in ranking the true classes higher. However, the species present in this example are insect sonotypes that were not present in the clean audio training set, meaning that the prediction is largely driven by  $\mathbb{P}(\text{class\_name})$  and contextual soundscape features. This highlights the advantage of the hierarchical approach in such cases, although it does not allow a definitive conclusion about which model better captures the decision boundaries of these sonotypes.



## 10 Conclusion

In this project, we explored audio classification for different species recognition using classical machine learning approaches. The work introduced us to audio datasets and a wide range of feature engineering techniques, ranging from standard spectral descriptors to more specialized handcrafted representations such as species profiles. We also investigated how image-processing methods, including Local Binary Patterns (LBP) and Gray-Level Co-occurrence Matrices (GLCM), can be adapted to spectrogram representations in order to characterize acoustic texture and temporal structure.

On the modeling side, we explored an approximation of the Multiple Instance Learning (MIL) framework for weakly labeled soundscapes. Compared with a conventional approach based on independent 5-second chunks, the MIL-inspired formulation produced substantially stronger results, particularly on soundscape recordings where species vocalizations are sparse and temporally localized. These results suggest that aggregating information across multiple windows is better suited to the non-stationary nature of environmental audio.

The project also highlighted several practical challenges related to scalability. Audio segmentation, especially in the MIL setting, produced nearly two million feature vectors, leading to significant memory and computational constraints. As a result, the initial logistic regression approach was replaced by the more scalable `SGDClassifier`, combined with custom mini-batch training, weighted logistic loss, and batched inference.

Overall, this project shows that soundscape classification can be effectively approached using handcrafted audio features combined with classical machine learning models. In particular, a multiple instance learning (MIL) formulation over 5-second segments appears to be a practical way to bridge the gap between segment-level predictions and soundscape-level labels without relying on deep learning architectures.

# Appendix

## A Soundscapes annotated timelines



Figure 40: Annotated timelines of selected soundscapes showing ground-truth species presence over time at the 5-second chunk level

## B Soft spectral Gating

| Category                                       | Configuration      |
|------------------------------------------------|--------------------|
| <b>Soft Spectral Gating Parameters</b>         |                    |
| Sampling rate                                  | 32,000 Hz          |
| FFT window size                                | 1024 samples       |
| Hop length                                     | 512 samples        |
| Noise percentile threshold                     | 15.0               |
| Attenuation factor ( $\alpha$ )                | 3.0                |
| Frequency smoothing kernel size                | 3                  |
| Temporal smoothing kernel size                 | 3                  |
| Numerical stability constant ( $\varepsilon$ ) | $1 \times 10^{-8}$ |

Table 8: Soft spectral gating configuration used for noise reduction in the preprocessing pipeline

## C Features

| Category                                        | Configuration            |
|-------------------------------------------------|--------------------------|
| <b>Audio Feature Extraction</b>                 |                          |
| Sampling rate                                   | 32,000 Hz                |
| FFT window size                                 | 1024 samples             |
| Hop length                                      | 512 samples              |
| Rolling normalization percentile                | 0.85                     |
| Mel filter banks                                | 128                      |
| Texture downsampling (frequency bins)           | 32                       |
| Texture quantization levels                     | 8                        |
| Texture patch length (frames)                   | 9                        |
| Local Binary Pattern bins                       | 8                        |
| Event detection window (frames)                 | 9                        |
| Event detection threshold (standard deviations) | 1.0                      |
| <b>Pooling Strategy</b>                         |                          |
| Pooling method                                  | Chunk-based aggregation  |
| Aggregation statistics                          | Mean, Standard deviation |
| Percentile features                             | 10th, 90th percentile    |
| <b>Chunking Configuration</b>                   |                          |
| Chunk duration                                  | 5.0 seconds              |
| Chunk step size                                 | 2.5 seconds              |
| <b>Multiple Instance Learning (MIL)</b>         |                          |
| MIL chunk duration                              | 1.0 seconds              |
| MIL step size                                   | 0.5 seconds              |

Table 9: Audio feature extraction, pooling, and chunking configuration used in the pipeline

## D Models parameters

### D.1 SGDClassifier (5-second chunk model)

| Category                  | Configuration  |
|---------------------------|----------------|
| <b>Model</b>              |                |
| Type                      | SGDClassifier  |
| Loss                      | log_loss       |
| Penalty                   | elasticnet     |
| Alpha                     | 1.0e-4         |
| Max Iterations            | 3000           |
| Learning Rate             | optimal        |
| Averaging                 | true           |
| Random State              | 42             |
| <b>Training</b>           |                |
| Preprocessing             | StandardScaler |
| $\beta$ (class weighting) | 0.5            |
| Batch Size                | 1024           |
| Epochs                    | 20             |
| Early Stopping            | true           |
| No Improvement Iterations | 4              |
| Tolerance                 | 0.0001         |

Table 10: Model and training configuration for the 5-second chunk model

### D.2 SGDClassifier (MIL model)

| Category                  | Configuration                                       |
|---------------------------|-----------------------------------------------------|
| <b>Model</b>              |                                                     |
| Type                      | SGDClassifier                                       |
| Loss                      | log_loss                                            |
| Penalty                   | elasticnet                                          |
| Alpha                     | 1.0e-4                                              |
| Max Iterations            | 3000                                                |
| Learning Rate             | optimal                                             |
| Averaging                 | true                                                |
| Random State              | 42                                                  |
| <b>Training</b>           |                                                     |
| Preprocessing             | StandardScaler                                      |
| $\beta$ (class weighting) | 0.5                                                 |
| Batch Size                | 4096                                                |
| Epochs                    | 5                                                   |
| Early Stopping            | true                                                |
| No Improvement Iterations | 1 (reduced to speed up training in the MIL setting) |
| Tolerance                 | 0.0001                                              |

Table 11: Model and training configuration for the MIL model

### D.3 MIL probability denoiser

| Stage          | Configuration      |
|----------------|--------------------|
| Preprocessing  | StandardScaler     |
| Model          | LogisticRegression |
| Max iterations | 1000               |
| Solver         | liblinear          |
| Class weight   | balanced           |

Table 12: Logistic regression configuration used as probability denoiser in the MIL pipeline

### D.4 Soundscape probability calibrator

| Parameter            | Value         |
|----------------------|---------------|
| Model type           | XGBClassifier |
| Number of estimators | 300           |
| Maximum depth        | 3             |
| Class weight         | balanced      |

Table 13: XGBoost configuration used for soundscape probability calibration